

中国研究生创新实践系列大赛
“华为杯”第二十二届中国研究生
数学建模竞赛

题 目 机器学习驱动的面向 MIMO-OFDM 的链路速率预测

摘 要：

无线通信系统是支撑现代社会连接的基础设施，其链路速率是评判通信质量的一个重要指标。TxBF（发送波束赋形）被视为在 MIMO（多输入多输出）系统下获得额外传输速率提升的信号处理方法，理论上，在给定带宽和信噪比条件下，信道能够无差错传输的最大链路速率只受限于香农容量。但在实际的系统中，链路速率受多种因素影响，包括编码方式、调制方式、噪声与干扰等，目前尚无一种较好的方法能够对实际链路速率进行精确的建模。为此，本文使用了多种方法对不同处理方法下的链路速率进行建模。

针对问题一，在关闭 TxBF 的情况下，依据物理层抽象的通用流程，先逐子载波单元计算信干噪比 $SINR_k$ 并提取 $SINR_k$ 序列的统计与结构特征。接着以 EESM 模型为核心，通过题目中所给的完备训练集对 EESM 模型中的重要参数 α 和 β 进行拟合，得到最优的结果，并采用 Isotonic 校准器对等效信噪比与传输速率的映射关系进行概率与阈值校准，并利用交叉验证生成 OOF（out-of-fold）预测以抑制信息泄漏，并构造可靠的基线（ISO）输出；在此基础上，继续进行增强建模，构建堆叠特征，训练梯度提升回归（HGBR）以捕捉非线性与高阶交互，并通过学习型加权对 ISO 与 HGBR 进行融合，生成兼具可解释性与表征力的预测模型。最终改进后的模型指标为 $R^2 = 0.828$ ， $RMSE = 40.657$ 。并以此模型对关闭 TxBF 情况下的不完备数据集 B 进行预测。

针对问题二，在关闭 TxBF 的情况下，本文提出了一种基于数据驱动的新型速率估计模型。首先，同问题一，围绕 122 个子载波单元计算其信干噪比 $SINR_k$ ，并转化为线性域和 dB 域，对其中构建统计特征（如均值方差、标准差等）。由于无线通信信道具有高度的

非线性和相关性，因此本文选择使用随机森林回归模型来抓住其中的非线性和相关性，将 122 个子载波单元计算其信干噪比 $SINR_k$ 的线性域和 dB 域以及统计特征构建成一个具有 259 维的高纬度特征作为随机森林的训练特征，训练阶段在组内采用 GroupKFold 划分以避免同域泄漏，并对标签长尾采用等频 10 分箱的逆频权重，进行随机搜索寻优，并在每一折内对训练折的输出拟合 Isotonic 校准器后作用于验证折，最终用全训练集重训并得到校准后的概率/分数用于连续值到离散值的最近映射。最终该随机森林回归模型的指标为 $R^2 = 0.765$ ， $RMSE = 47.557$ 。并以此模型对关闭 TxBF 情况下的不完备数据集 B 进行预测。

针对问题三，在开启 TxBF 的情况下，本文结合物理层抽象的通用模型和数据驱动的方法，提出了一种新型速率估计模型。首先，利用 2×2 闭式特征值公式在每个子载波上提取 SVD 结构量，并以此得到等效功率与 $SINR_k$ 序列，作为后续建模的物理基础。随后通过 EESM 将多载波 $SINR_k$ 压缩为 $SINR_{eff}$ ，并对 β 采用粗细搜索得到最优 β^* ，并学习 $SINR_{eff}$ 到传输速率的单调映射。随后，根据拆解 SVD 得到的 4 维统计特征与 dB/线性域统计特征、多 β -ESSM 特征，底噪信息构建出一个 30 维的特征，以此训练 HGBR，最终以带岭正则的闭式将两种模型进行融合。最终该模型的 $R^2 = 0.702$ ， $RMSE = 62.195$ ， $Top-1 = 0.391$ ， $\pm 1 = 0.861$ 。并以此模型对开启 TxBF 情况下的不完备数据集 B 进行预测。

关键词：EESM；随机森林；HGBR；带岭正则闭式；
Isotonic 校准器

目 录

一、 问题重述	5
1.1 问题背景	5
1.2 问题的提出	5
二、 问题分析(画总体技术路线图).....	5
2.1 问题一分析	6
2.2 问题二分析	6
2.3 问题三分析	7
三、 基本假设与符号说明	8
3.1 基本假设	8
3.2 符号说明	8
四、 数据观察与概述	10
4.1 数据来源说明	10
4.2 notxbf_com_excels_f4 数据集宏观统计与可视化	11
4.2.1 数据集宏观统计	11
4.2.2 数据集可视化	12
4.3 txbf_com_excels_f4 数据集宏观统计与可视化.....	17
4.3.1 数据集宏观统计	17
4.3.2 数据集可视化分析	17
五、 问题一的求解	23
5.1 问题分析	23
5.2 数据预处理	24
5.3 模型建立	24
5.3.1 指数型 ESM (EESM) 模型	24
5.3.2 Isotonic Regression 映射模型.....	25
5.3.3 HGBR 回归模型	26
5.3.4 HGBR 堆叠融合 (ISO+HGBR)	26
5.3.5 整体算法流程图	26
5.4 模型求解	29
5.5 结果分析	30
5.5.1 堆叠模型对等效 SNR - MCS 关系的精准拟合与稳健表现	32
5.5.2 堆叠融合模型的误差结构与校准性评估	32
5.5.3 堆叠映射显著提升 MCS 拟合一致性.....	33
六、 问题二的求解	34
6.1 问题分析	34
6.2 数据预处理	34
6.2.1 基本的清洗与筛选	34
6.2.2 信道状态信息 CSI 解析与子载波单元信噪比 SINR 计算	35
6.2.3 特征构建与附加统计量	35
6.3 随机森林回归模型建立	36
6.4 模型求解	37
6.5 模型结果与分析	39

七、 问题三的求解	43
7.1 问题分析	43
7.2 数据预处理	44
基本的清洗与筛选	44
7.2.1 信道状态信息 CSI 解析与子载波单元信噪比 SINR 计算	44
7.2.2 等效 SNR 与特征构建	44
7.2.3 特征构建	44
7.3 融合模型建立于求解	46
7.3.1 物理模型	46
7.3.2 ML 模型	46
7.3.3 融合模型与二次校准	47
7.4 模型结果与分析	48
八、 模型评价与改进	54
8.1 模型优点	54
8.2 模型缺点	54
8.3 模型的改进与推广	54
参考文献	56
附录	57
附录 1: 附件文件及内容	57
附录 2: 问题一关键代码	57
附录 3: 问题二关键代码	66
附录 4: 问题三关键代码	72

一、问题重述

1.1 问题背景

自 1896 年马可尼首次实现无线电通信以来，无线通信技术经历了多次技术革新，相较于初期，如今的 Wi-Fi 6 在传输速率上已提升 873 倍，完成了由军事专用到普惠生活的转变。在现代无线通信中，OFDM（正交频分复用）调制技术与 MIMO（多输入多输出）技术被广泛应用于提升无线传输速率。MIMO-OFDM 系统通过利用空间复用以及频分复用的方法，允许通过多个天线同时发送和接收数据，显著提高了无线信道的容量^[1]。

随着网络的普及，无线通信设备与无线终端逐渐多样化，进而导致信道条件和网络负载情况复杂度大幅提升，故而需要进一步估计与优化链路速率，达到系统整体性能提升的目的。EESM (Exponential Effective SINR Mapping) 模型常被用于无线通信系统中估算信道容量和传输速率，它通过将复杂的多载波系统中的各个信道的信干噪比 (SINR) 映射到一个有效的 SINR 以进行降维操作，从而简化了信道速率的计算^[2]。然而，在复杂和动态的无线环境中，EESM 模型的适应能力还需进一步提升。

1.2 问题的提出

基于上述问题背景，本问题的核心是利用实际数据，优化无线通信系统的信道传输速率估计，实现对链路速率的较为准确的预估。

问题一：EESM 模型参数拟合与链路速率估计

在关闭 TxBF（发送波束赋形）的情况下，利用赛题中提供的完备数据集 A 中的实际传输 MIMO 信道数据以及传输速率 $rate$ （标签）对 EESM 模型进行参数拟合，并使用拟合后的 EESM 模型预测不完备数据集 B，计算出每个载波单元的 SINR 并进行等效 SINR 映射，得到信道传输速率 $rate$ 。

问题二：提出新型速率估计模型

在关闭 TxBF（发送波束赋形）的情况下，利用赛题中提供的完备数据集 A 中的实际传输 MIMO 信道数据以及传输速率 $rate$ （标签）设计并训练一种新型速率估计模型，再将训练好的模型应用于不完备数据集 B 中的信道状态信息，进行信道速率预测。

问题三：

在开启 TxBF（发送波束赋形）的情况下，利用赛题中提供的完备数据集 A 中的实际传输 MIMO 信道数据以及传输速率 $rate_{txbf}$ 设计并训练一种新型速率估计模型，再将训练好的模型应用于不完备数据集 B 中的信道状态信息，进行信道速率预测。

公式章 (下一章) 节 1

二、问题分析(画总体技术路线图)

本文技术路线如下：

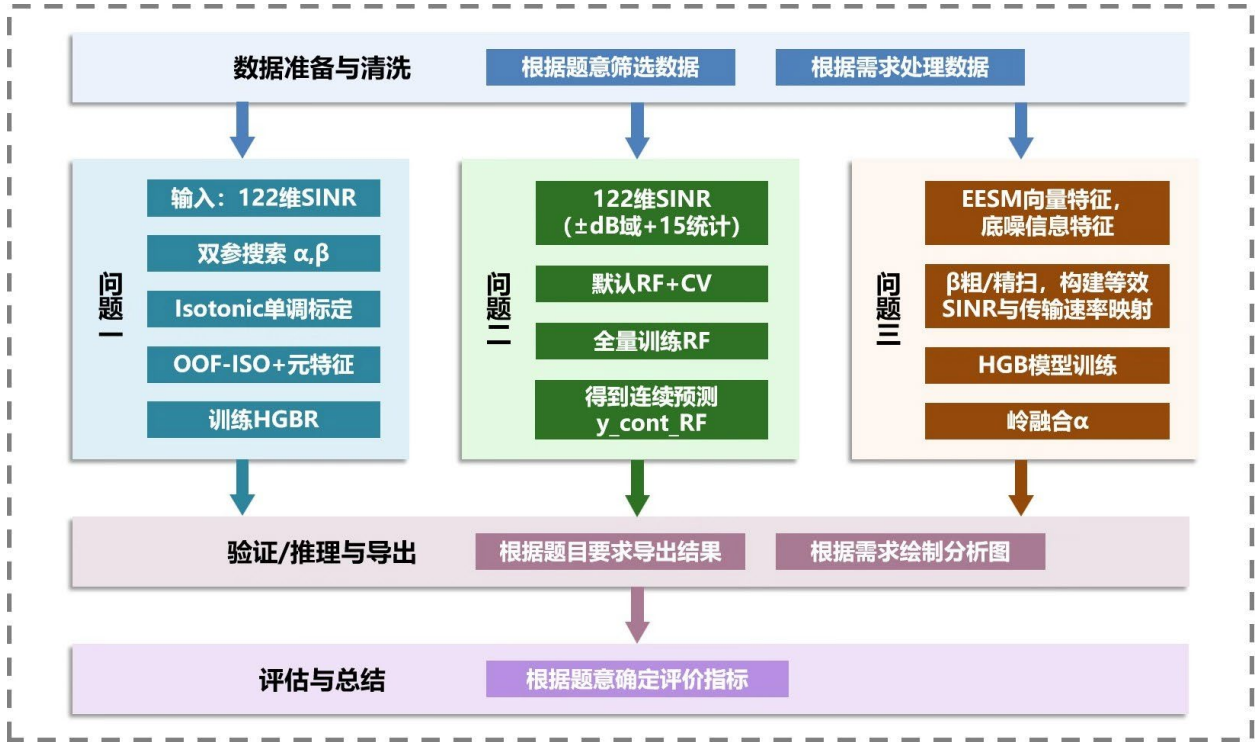


图 2-1

2.1 问题一分析

问题一要求进行 EESM 模型参数拟合与链路速率估计，主要分为两部分，包括提出拟合结果表现出尽可能高的准确度的 EESM 模型以及利用拟合出的 EESM 模型得到不完备数据集 B 中各信道的传输速率。

首先，针对完备数据集 A 中的每个载波单元计算对应的 SINR（信干噪比）；再利用得到的 $SINR_i$ 计算等效 SINR 映射 $SINR_{eff}$ ，通过数据集 A 中实际传输速率和 SINR 的对应关系，使用 EESM 模型来拟合模型参数^[3]。完成模型参数的拟合后，使用已经训练好的 EESM 模型，基于不完备数据集 B 中提供的 CSI 数据，计算每个载波单元的 $SINR_i$ 并进行等效 SINR 映射，再对其信道状态信息进行链路速率的估算。

2.2 问题二分析

问题二要求提出新型速率估计模型用于对不完备数据集 B 进行传输速率估计。为满足问题中所要求的关闭 TxBF 条件，首先是数据预处理与特征工程步骤，先对数据进行筛选，保留满足题目要求且有效的数据，并计算得到完备数据集 A 中每个载波单元对应的 $SINR_i$ 并提取训练样本特征；随后，设计并训练新模型，本文采用随机森林模型，利用前序步骤得到的数据特征作为训练特征；最后，使用训练好的模型对不完备数据集 B 中的信道信息进行传输速率预测，并通过 OOB（Out-of-Bag）评估来计算模型的评估指标。

2.3 问题三分析

问题三要求提出新型速率估计模型用于对开启 TxBF 条件的不完备数据集 B 进行传输速率估计。同问题二，需要对题目所提供的开启 TxBF 的完备数据集 A 进行数据处理并提取相关的特征作为训练样本特征；随后，设计并训练新模型，本文利用 EESM 来计算等效 SINR（即 $SINR_{eff}$ ），对 EESM 中的 β 进行粗细扫描寻优得到最优的 β^* ，构建从 $SINR_{eff}$ 到传输速率的映射 g_{iso} ，同时结合数据驱动方法，采用 HGB 模型对构建的多维度特征进行回归学习，构建从数据特征到传输速率的映射 g_{ml} ，将两种模型按岭化闭式解求得的 α 进行线性融合模型；最后，使用该融合模型对开启 TxBF 条件的不完备数据集 B 中的信道速率进行预测。

公式章 (下一章) 节 1

公式章 (下一章) 节 1

三、基本假设与符号说明

3.1 基本假设

假设 1：在 SINR 的计算中，每个载波单元的噪声设为总噪声的平均值；

假设 2：csi_time（信道信息时戳）与 dfx_time（速率时戳）对齐且有效，两者时间差落在信道相干时间内；

假设 3：在无断层异常的情况下，等效 SINR 与 MCS 单调不降。

3.2 符号说明

符号表示	含义说明
B	无线信道总带宽（Hz）
K	载波单元数量，求解中取 $K = 122$
y_i	第 i 个载波单元上的接收信号
h_i	第 i 个载波单元上的无线信道系数
s_i	发送信号
n_i	接收机处的噪声
N	接收天线数量，求解中取 $N = 2$
M	发送天线数量，求解中取 $M = 4$
$h_{n,m}^i$	在第 i 个载波单元上，第 m 根发送天线到第 n 根接收天线的信道系数
$\mathbf{H}_i$	MIMO-OFDM 中的信道系数矩阵
$\mathbf{V}_i$	SVD 分解结果，在第 i 个载波单元对应的 TxBF 加权矩阵（酉矩阵）
SINR_i	第 i 个载波单元的信干噪比
SINR_{eff}	等效 SINR，所有载波单元 SINR_i 映射得到
C	香农容量
p_i	SINR_i 的概率质量函数
$I(\bullet)$	度量函数，表征信道传输速率

EESM	指数信息度量函数
$\Phi(\bullet)$	ESM 映射函数
α	EESM 模型待拟合参数，缩放系数
β	EESM 模型待拟合参数，偏移系数

公式章 (下一章) 节 1

四、数据观察与概述

在本题中，实验数据量较为庞大，整体可划分为两类：关闭 TxBF 与开启 TxBF 条件下的无线通信系统链路数据。为了为后续建模与问题求解提供基础，本章首先对数据来源及其主要特征进行介绍，并结合初步分析结果建立对题目的整体认知。这一过程不仅为解题思路的形成提供支撑，同时也为数据预处理与模型设计奠定了必要的基础。

4.1 数据来源说明

为了保证问题数据能够真实反映无线通信系统的运行特性，所使用的问题数据均由现网环境下的无线基站采集完成。实验环境覆盖典型的无线通信场景，能够在不同位置条件下记录链路特征。数据采集过程中，网络设备始终保持对终端通信行为的监测与记录，从而确保数据具有连续性与完整性。

具体的采集流程如下：实验终端（手机）从起始位置（位置 1）开始，在该位置保持静止一段时间，以便网络设备能够充分收集该点位下的信道状态信息与链路速率信息。随后，终端自动移动至下一个目标点位（位置 2），并再次保持静止以进行数据采集。通过依次遍历所有预设点位并重复“移动—停留—采集”的过程，最终形成覆盖多个空间位置的数据集，采集的所有数据存储在 excel 表格中，包含上行传输过程中的信道状态信息矩阵、信道信息采集时间等。

在数据采集过程中，网络设备能够同步记录以下关键信息：

- 1) **信道状态信息（CSI）矩阵**：反映多天线、多子载波条件下的瞬时信道特征；
- 2) **链路传输速率（MCS 指标）**：对应实际的链路性能表现；
- 3) **终端索引（MAC 地址）**：用于区分不同测试终端；
- 4) **采集时间戳**：包括信道状态采集时间与速率信息采集时间，用于保证数据时序一致性；
- 5) **噪声信息（Noise Floor）**：反映采集过程中的底噪水平，为后续建模提供信噪比参考。

整个采集过程如图 4-1 所示，图中展示了移动终端从起点到各预设点位的移动轨迹与停留方式。通过在不同位置的重复采样，实验数据能够覆盖多样化的信道状态，从而为后续建模与速率预测提供坚实的数据支撑。最终的采集结果分别存放在 notxbf_com_excels_f4（TxBF 关闭）与 txbf_com_excels_f4（TxBF 开启）。

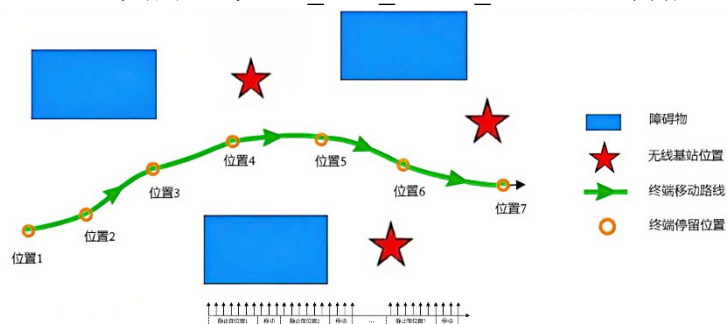


图 4-1 数据采集过程说明

4.2 notxbf_com_excels_f4 数据集宏观统计与可视化

4.2.1 数据集宏观统计

TxBF 关闭条件下的采集的实验数据集存放在文件夹 notxbf_com_excels_f4 下，该文件夹下有三个子文件夹使用终端 mac 地址命名，每个文件夹下包含训练数据集 A(Train)和测试数据集 B (Valid) 两个文件目录。训练数据集，包含了信道状态信息和传输速率(标签)，验证数据集仅包含信状态信息，具体的数据集分析与数据变量见表 4-1，表 4-2：

表 4-1 notxbf_com_excels_f4 数据集分析

文件夹	表格名称	样本数量
341c-f0d4-70be\train	341c-f0d4-70be_0.xlsx	7712
bc98-2983-907b\train	bc98-2983-907b_0.xlsx	6496
d0c1-bfe4-18f0\train	d0c1-bfe4-18f0_0.xlsx	6161
总计		20369
341c-f0d4-70be\valid	341c-f0d4-70be_0.xlsx	7712
bc98-2983-907b\valid	bc98-2983-907b_0.xlsx	6161
d0c1-bfe4-18f0\valid	d0c1-bfe4-18f0_0.xlsx	6496
总计		20369

表 4-2 notxbf_com_excels_f4 数据变量分析

列名	数据维度	含义(单位)	备注
csi_matrix_r0_c0 ~ csi_matrix_r1_c3	2*4*122	信道状态信息(-)	发送天线数 4，接收天线数 2，载波单元数量 122
csi_time	1	信道状态信息采集时间(ms)	信道信息采集时间
beamforming_en	1	波束赋型开启指示(-)	1 表示开启波束赋型，0 表示未开启
mcs	1	传输速率(-)	传输速率，valid 中数据集无此列
dfx_time	1	传输速率采集时间(ms)	速率信息采集时间
noise_floor	1	底噪信息(dBm)	信道底噪信息

采集的所有数据存储在 excel 表格中，包含上行传输过程中的信道状态信息矩阵、信道信息采集时间，其中信道状态信息矩阵被切分后存储在列 csi_matrix_r0_c0~csi_matrix_r1_c3 中，其中 r 表示信道矩阵的行号，c 表示信道矩阵的列号。表格中每个单元包含了 122 个载波单元的信道信息。另外 excel 中还记录下行传输过程中对应的传输速率 mcs，传输速率采集时间 dfx_time，信道状态信息采集时间 csi_time 与波束赋型开启指示 beamforming_en。另外，noise_floor 表示传输过程中的底噪。

4.2.2 数据集可视化

本节对 Train 与 Valid 的图谱（频域画像/热力图、8 路相关、噪声地板、平均 SINR、SVD、MCS 及其与 SINR/噪声的关系）做并列速览：Valid 频率选择性更强（起伏与凹槽更深更宽），但噪声更集中；两组均呈稳定的发射端口指纹（列内形状相近，Tx 差异主导于 Rx）； H_{avg} 低秩（ $\sigma_1 \gg \sigma_2$ ），平均 SINR 与 MCS 近单调正相关、噪声仅次要；MCS 分布长尾不均衡，样本集中在中等等级。工程含义：Valid 侧均衡与编码交织需更激进，MCS 自适应更保守；双流传输应择优在 σ_2 相对抬升的时频块启用；对 Train 的高噪声/高 SINR 尾部做稳健剪裁，并在建模与评估中采用类权重/分层统计以对冲类别不均衡。

图 4-2 所示的 8 路 $|H|^2$ dB 曲线中，Train 的整体更平坦，c1 列（r0c1/r1c1）几乎水平、其余链路多为轻微斜率或中等波纹；而 Valid 的起伏显著增大，c3 列（r0c3/r1c3）出现更深的频域凹槽与大尺度弯曲，体现更强的频率选择性。两张图在“同一列”上的形状相似、行间差异较小，说明发射端口指纹主导（Tx 链路差异 > Rx 差异）；Train→Valid 的变化更像由场景/姿态或干扰增强引起。结论：Valid 比 Train 更“陡更坑”，坏子载波更多，均衡与编码交织压力更大，自适应 MCS 需更保守。

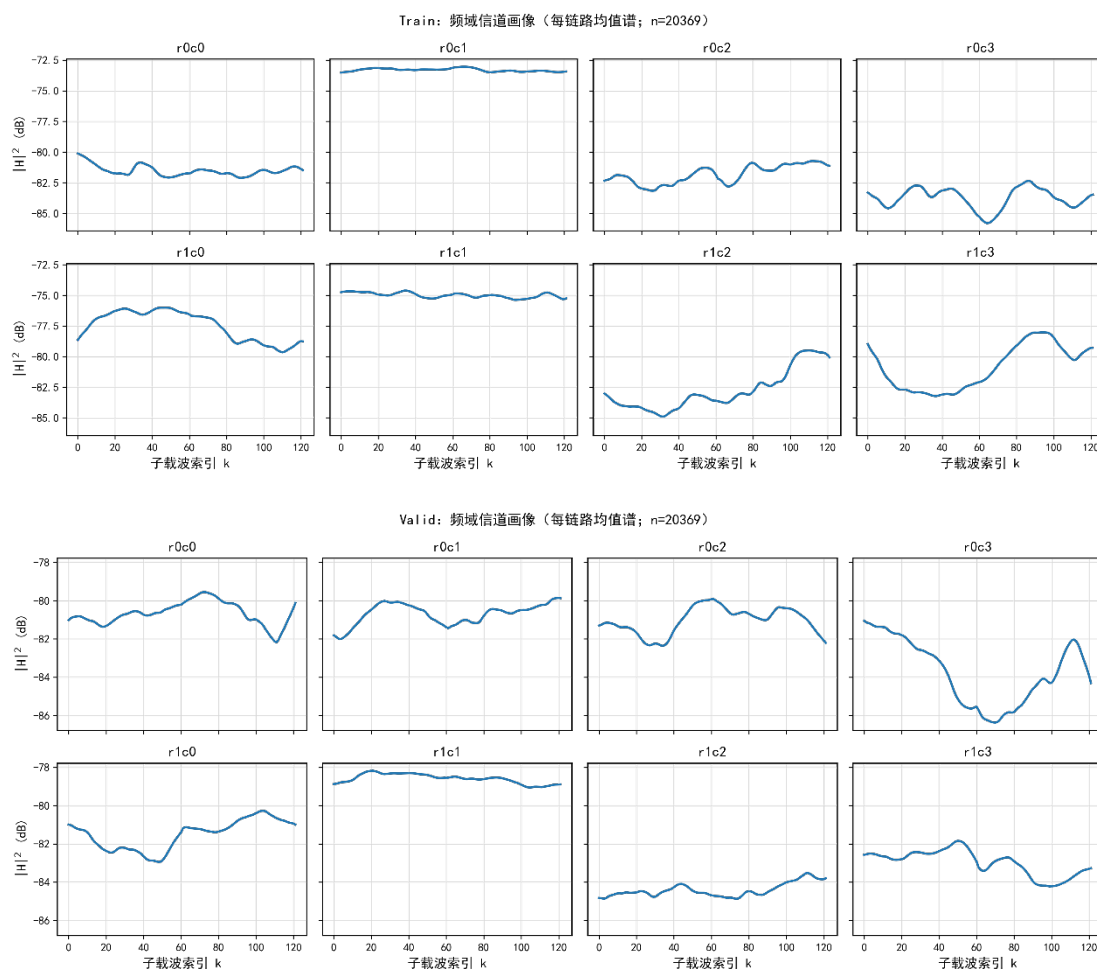


图 4-2 8 路 $|H|^2$ dB 曲线

图 4-3 所示的频域信道画像热力图中（横轴为 122 个子载波，纵轴为 8 条 r0c0...r1c3 链路，颜色表示 $|H|^2 / |H|^2 \text{dB}$ ），Train 中各行多为近似水平色带，仅有缓慢梯度与浅窄暗槽，说明频率选择性较弱；Valid 则颜色起伏更大、出现更宽更深的暗槽和明显左右梯度，表明频率选择性显著增强。两图在同一“行”的亮度与形状整体相似，显示出稳定的发射端口指纹（Tx 链路差异主导于 Rx 差异）。工程含义：Valid 相比 Train 需要更强的均衡与编码交织，自适应 MCS 需更保守，并宜在预编码/调度上对深衰子带做规避或分组。

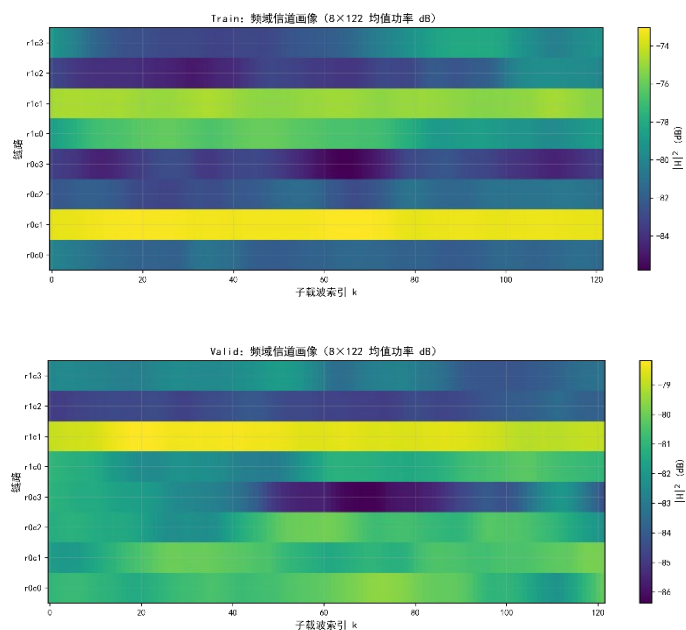


图 4-3 频域信道画像热力图

图 4-4 所示的是 8 路链路均值相关性热力图（元素为各链路均值 $|H|^2 / |H|^2$ 的 Pearsonr）：Train 中整体相关性高、块状结构明显（多在 0.6–0.95），如同一发射端口跨接收端的相关性很强（如 r0c1–r1c1≈0.95），体现稳定的 Tx 端口指纹；Valid 仍保留该指纹但整体相关性略降，并出现低相关“异常链路”——r1c3 与多数链路仅~0.27–0.42，r0c1–r1c1 也降至~0.62，说明场景/姿态变化使空间相关性减弱、有效秩倾向上升。工程含义：Valid 场景下预编码可利用的端口间冗余减少，需关注端口校准与对低相关/异常链路的权重抑制或分组处理。

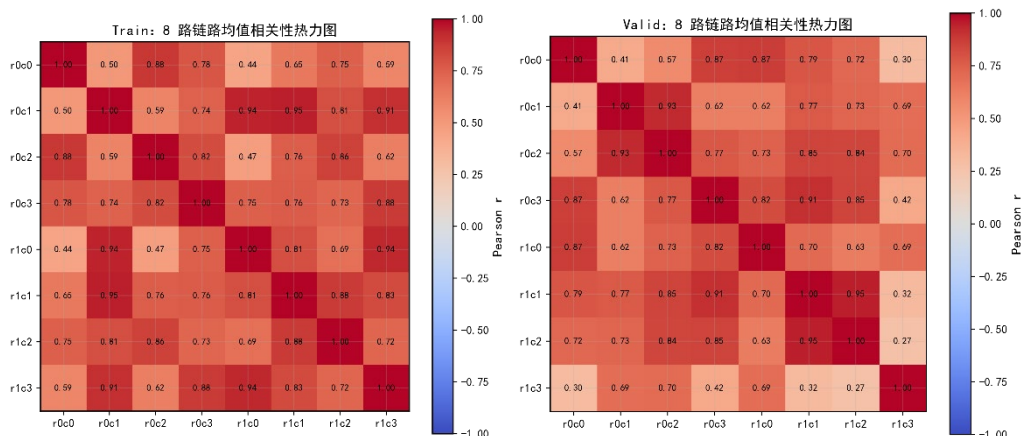


图 4-4 8 路链路均值相关性热力图

图 4-5 所示的是噪声地板分布图（单位 dBm）：两组都呈单峰、长右尾形状，主峰约在-94~-92dBm 附近。Valid 的直方与 KDE 明显更“瘦”，说明噪声更集中、波动更小；Train 分布更宽，右侧（噪声偏高、数值更接近 0dBm）尾部更厚，并出现少量高噪声离群点（可达-8x/-7xdBm 甚至更高），可能由瞬时干扰/采样场景差异或个别测量异常导致。工程含义：与 Train 相比，Valid 整体更安静、SINR 基线略优；后续建模/评估时建议对高噪声尾部做稳健处理（如剪裁到 P99 或 3σ 外点剔除），以避免少量极端样本放大小区间统计和可视化的偏差。

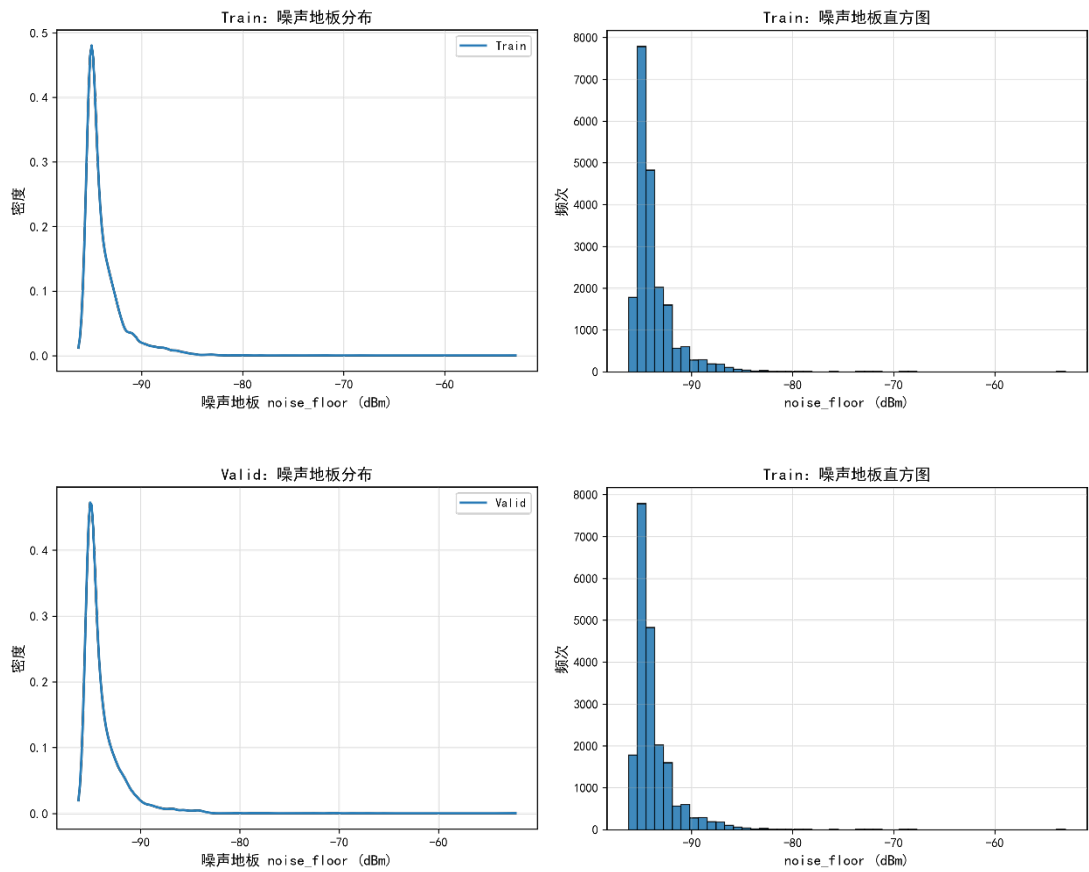


图 4-5 噪声地板分布图

图 4-6 所示的是每条样本的平均 SINR（dB）分布及其按文件的箱线图：从 KDE 看，Train 与 Valid 都是单峰、右长尾分布，峰值约在 7 - 10dB 附近；相比之下 Valid 的峰更尖、更集中，而 Train 的分布更宽，既有更多低于 0dB 的低值，也出现>30dB 的高值长尾，说明 Train 场景更“杂”、异质性更强。文件维度上，两组的中位数都在~8 - 12dB，Valid 各文件箱体更紧、离群点更少；Train 若干文件的须更长、离群更多，提示数据质量/场景差异或干扰波动更大。工程含义：Valid 的链路质量更稳定；对 Train 侧建议启用稳健统计（剪尾/截断）与自适应门限，以减小极端样本对建模与评测的影响。

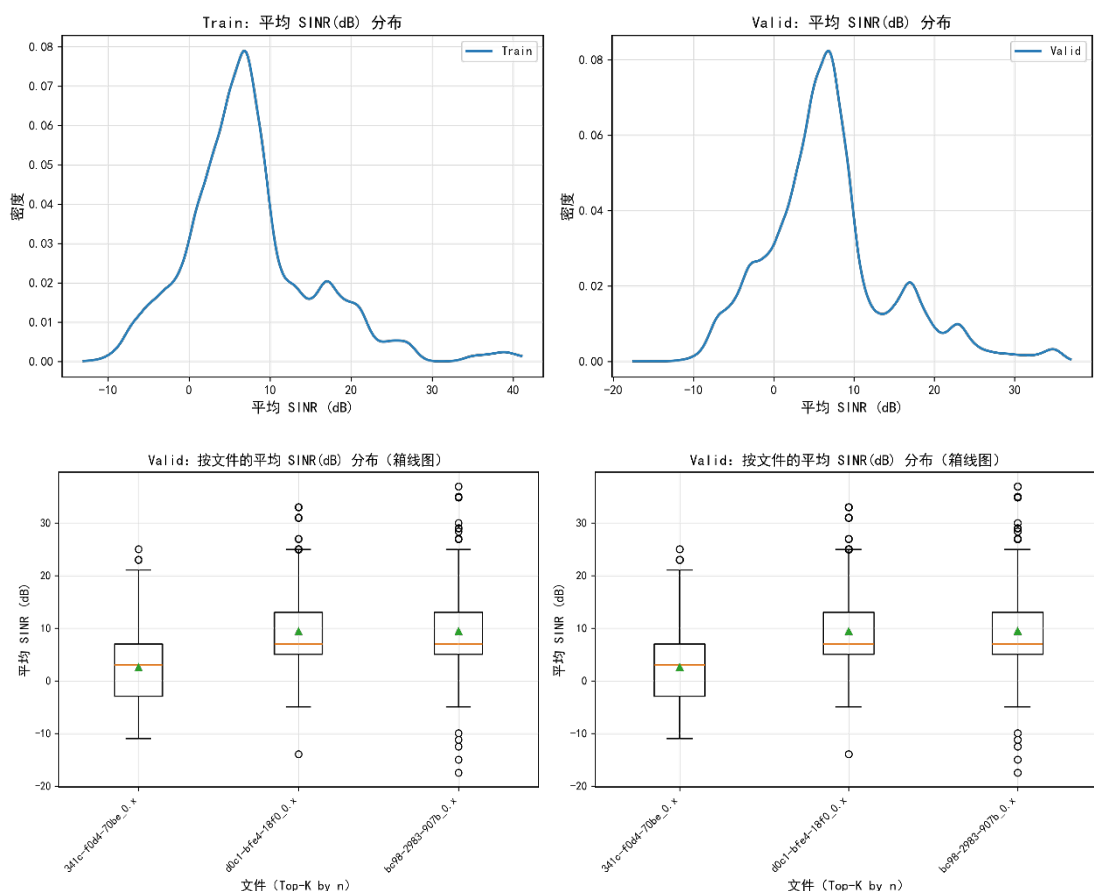


图 4-6 样本平均 SINR (dB) 分布及箱线图

图 4-7 所示的是相关散点图：左图是噪声地板 `noise_floor(dBm)` 与 MCS，右图是平均 SINR(dB) 与 MCS；蓝点为样本，红虚线为按横轴分箱后的中位数趋势。可以看到噪声 $\rightarrow$ MCS（左）：整体呈负相关，噪声升高（数值趋近 0dBm）时 MCS 明显下降，但低噪声区仍有低 MCS 的点，说明“噪声”不是决定因素，还受信号功率/频率选择性/波束状态等影响，趋势略显非单调。SINR $\rightarrow$ MCS（右）：趋势几乎单调上升，是更可靠的一维指标：约 0–5dB 区间 MCS 低、10–20dB 快速提升、>25dB 逐步趋于饱和（顶部阶跃由离散 MCS 限制造成）。

工程含义：用平均 SINR 做自适应调制的主控更合适，噪声可作为辅助（或用于异常检测/剪裁），并在高 SINR 区设置饱和上限与在低/中 SINR 区设置滞回以稳住切换。

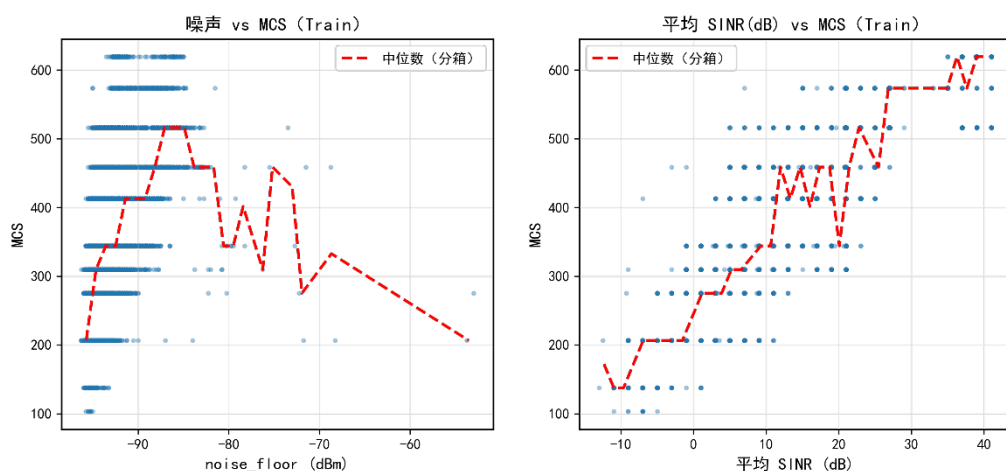


图 4-7 相关散点图

图 4-8 所示的分别是 MCS 降序条形图（含 Top-k 覆盖率）和离散 MCS（0-10）频次图。训练集呈明显长尾不平衡：少数几个中等 MCS 占了大部分样本，标题中的 Top-k 指标显示 Top-3 已覆盖过半、Top-5 基本覆盖大多数；高阶 MCS（右端）以及极低阶样本都很少。后续建模/评估需做类别不平衡处理（类权重/重采样/分层评估），并在回归→离散映射或阈值设定时做边界校准，避免稀有高阶 MCS 被主流类别“吞没”。

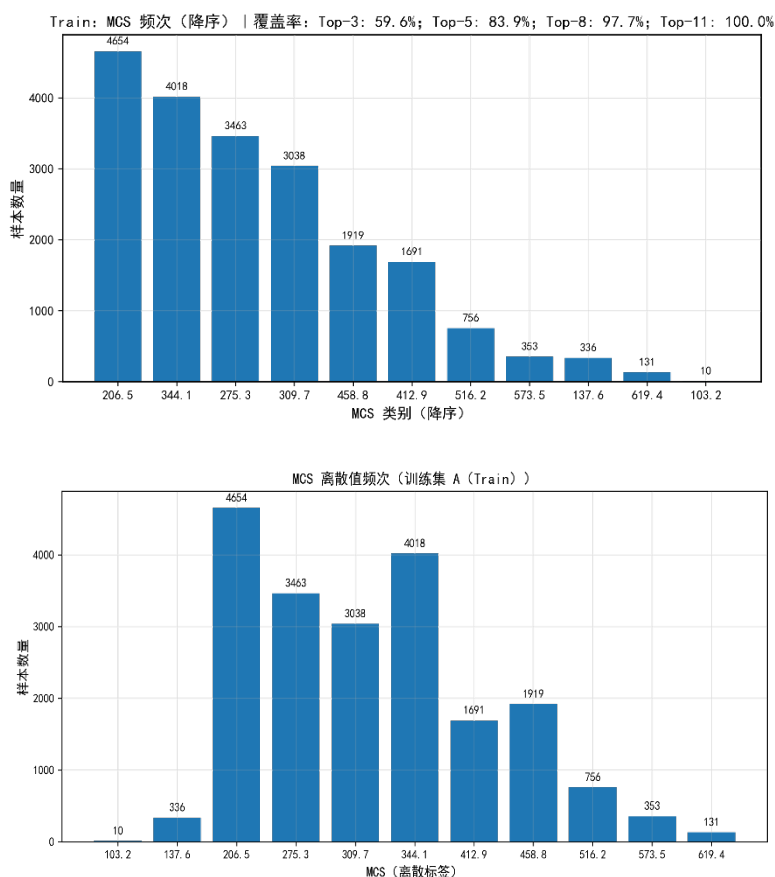


图 4-8 MCS 分布统计图

4.3 txbf_com_excels_f4 数据集宏观统计与可视化

4.3.1 数据集宏观统计

TxBF 开启条件下的采集的实验数据集存放在文件夹 txbf_com_excels_f4 下，该文件夹下有六个子文件夹使用终端 mac 地址命名，每个文件夹下包含训练数据集 A(Train)和测试数据集 B (Valid) 两个文件目录。训练数据集，包含信道状态信息和传输速率(标签)，验证数据集仅包含信状态信息，具体的数据集分析见表 4-3，数据变量与 4.2.1 节所述一致。

表 4-1 txbf_com_excels_f4 数据集分析

文件夹	表格名称	样本数量
341c-f0d4-70be\train	341c-f0d4-70be_0.xlsx	5034
508e-4963-d5dc\train	508e-4963-d5dc_0.xlsx	5290
508e-4963-e122\train	508e-4963-e122_0.xlsx	4168
bc98-2983-907b\train	bc98-2983-907b_0.xlsx	3709
d0c1-bfe4-18f0\train	d0c1-bfe4-18f0_0.xlsx	4697
fc02-9668-e2fd\train	fc02-9668-e2fd_0.xlsx	4651
总计		27549
341c-f0d4-70be\valid	341c-f0d4-70be_0.xlsx	5034
508e-4963-d5dc\valid	508e-4963-d5dc_0.xlsx	5290
508e-4963-e122\valid	508e-4963-e122_0.xlsx	4168
bc98-2983-907b\valid	bc98-2983-907b_0.xlsx	3709
d0c1-bfe4-18f0\valid	d0c1-bfe4-18f0_0.xlsx	4697
fc02-9668-e2fd\valid	fc02-9668-e2fd_0.xlsx	4651
总计		27549

相较于 notxbf_com_excels_f4 (TxBF 关闭) 下的数据集，txbf_com_excels_f4 (TxBF 开启) 在宏观上来看数据集文件数更多，总体的样本数据也有所增加，数据信息中 beamforming_en = 1，其余信息一致。

4.3.2 数据集可视化分析

本节对 TxBF 开启条件下的 Train 与 Valid 的图谱（频域画像/热力图、8 路相关、噪声地板、平均 SINR、SVD、MCS 及其与 SINR/噪声的关系）做并列速览：
图 4-9 所示的“频域信道画像（每链路均值谱）”：每个小子图对应 2×4 中一条 Rx-Tx 链路在 122 个子载波上的平均 $|H|^2$ （dB）。整体看，Train 的曲线起伏更明显、频选择性更强：上排几条在中高频处出现波峰/波谷（如右两图逐步上扬后回落），下排的 r1c0 呈整体下坡、r1c3 有“U 形”低谷，链路间形态差异较大。Valid 则更平滑稳定：多数链路呈缓慢单调变化（如 r0c2、r0c3 逐步上升，r1c1 缓升，r1c3 缓降），波动幅度和链路间差

异相对更小。对比说明训练集环境更“花”、频率选择更强，而验证集更“稳”，可能对应采集场景或天线指向差异。

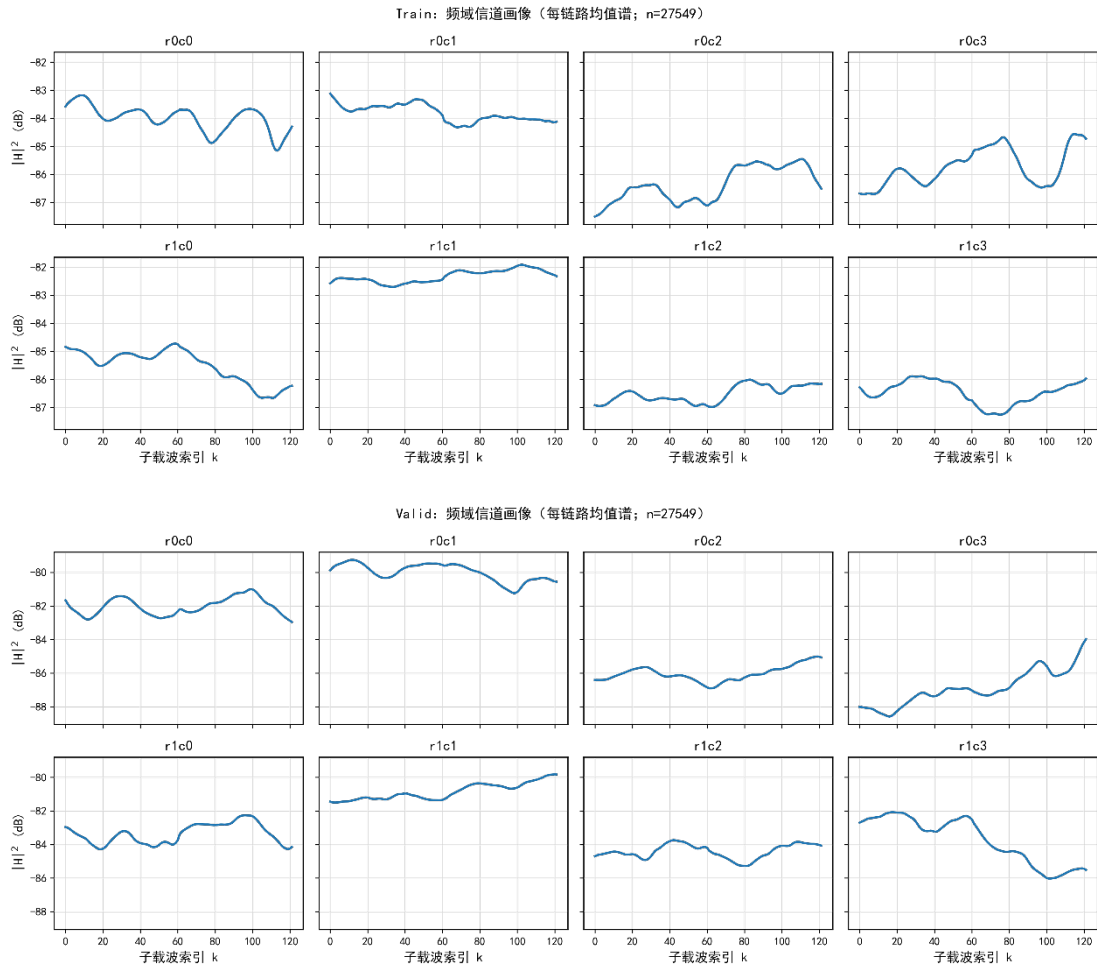


图 4-9 8 路 $|H|^2$ dB 曲线

图 4-10 所示的是“频域信道画像热力图”（8 条链路 $\times$ 122 个子载波，颜色表示各链路在每个子载波上的平均 $|H|^2$ 强度，近黄更强、近紫更弱）。整体上，Train 与 Valid 的结构相似：少数链路在全频段明显更亮，说明其长时平均增益更高；多数链路随频率只呈缓慢起伏，体现出温和的频率选择性。Train 图里个别链路在高、低子载波端出现更清晰的“斜坡”（强度单调增/减），频率选择性略更强；Valid 图的色彩过渡更平滑、对比度略低，显示各链路增益更为均匀。综合看，二者统计特性一致但场景差异导致的链路主次和频率选择性强弱仍可分辨。

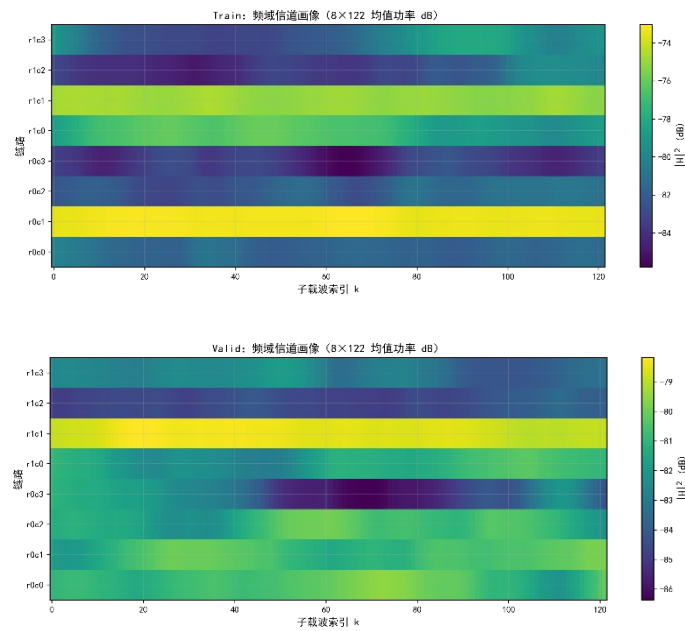


图 4-10 频域信道画像热力图

图 4-11 所示的是“8 路链路均值相关性热力图”（Train / Valid）。整体看，两套数据都呈现显著正相关，大多数成对相关系数落在 0.6–0.95，说明 8 路链路的平均功率随共同环境因素联动、信息冗余较高。Train 图中相关性更分散一些，诸如 r0c2 与 r0c3、r1c0 等只有约 0.46–0.67，显得相对独立；而 r0c0–r1c0、r1c1–r1c2 等组合接近 0.9–0.97，为最强耦合。Valid 图整体更“红”，大多数配对 $r \geq 0.8$ ，但仍存在 r0c2 这类弱相关“洼地”（与多路仅 ~ 0.49 –0.61）。因此：可以考虑基于相关性的链路选择/降维来抑制冗余，同时保留像 r0c2 这类相对独立的链路以提升判别力。

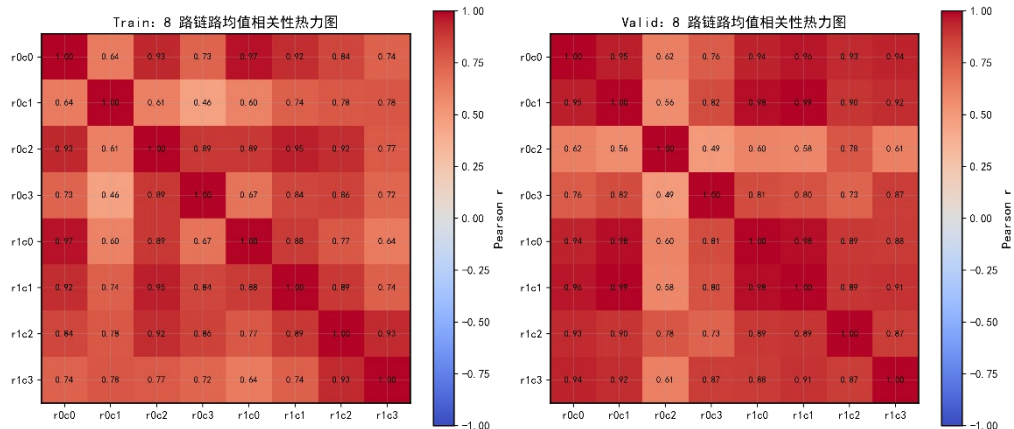


图 4-11 8 路链路均值相关性热力图

图 4-12 所示的是噪声地板（noise_floor, dBm）的 KDE 与直方图对照：两套数据都呈双峰（约落在 -96 与 -94 dBm）并伴随右长尾，说明存在两类典型底噪场景，同时偶发更高噪声片段；其中 Valid 更“瘦且稳”（方差更小、尾部更短），而 Train 分布更宽、右尾更厚，出现少量高噪声异常点。含义：Valid 的底噪更一致、基线更优；对 Train 建议做稳健处理（如截断至 P99/异常点剔除）并按场景或设备做分组标定，以降低极端样本对后续统计与建模的影响。

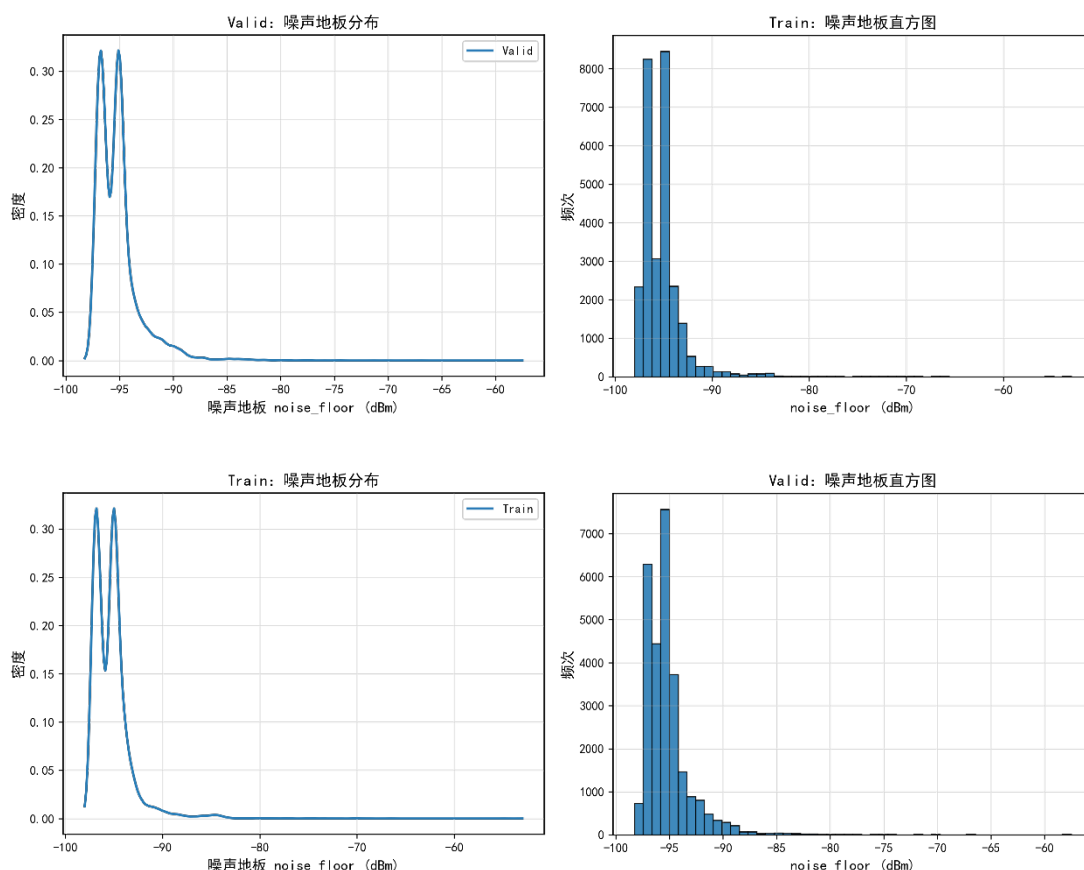


图 4-12 噪声地板分布图

图 4-13 所示的是平均 SINR(dB)的整体分布与按文件的离散度。从 KDE 看，Valid 的分布更尖且集中，主峰约在 6 - 9dB，而 Train 更宽且偏多峰，在 0dB 附近有肩峰并向高 SINR 拉出长尾，说明训练数据场景更杂、波动更大。按文件的箱线图进一步印证：Train 的文件间差异更显著（IQR 更宽、离群点更多），Valid 相对一致、但不同文件中位数仍存在差距。工程含义：后续建模/评测需做分文件/分场景分层与稳健指标（如中位数、P90）报表；训练时可考虑按文件重采样/权重以缓解分布移位，并在推理侧加入自适应门限应对文件级偏差。

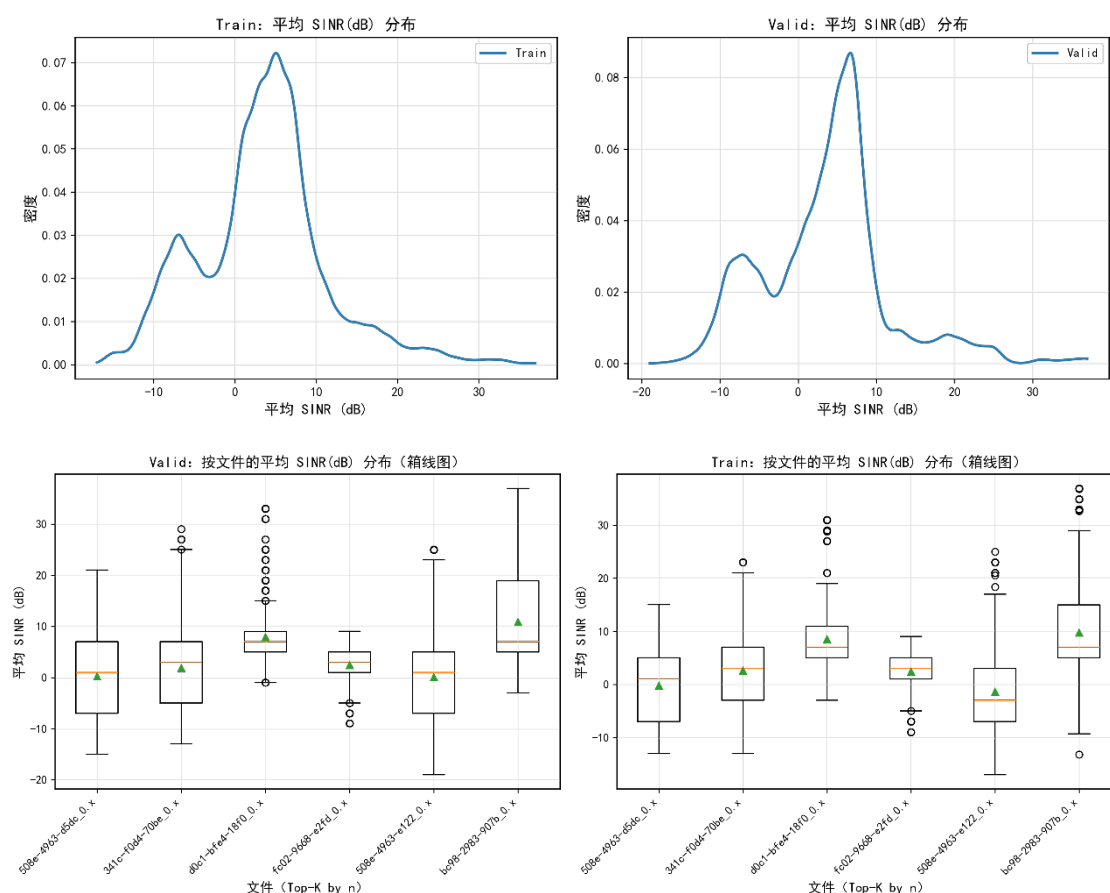


图 4-13 样本平均 SINR (dB) 分布及箱线图

图 4-14 所示的是“散点+分箱中位数”关系图：左图为噪声地板（noise_floor）vsMCS，右图为平均 SINR(dB)vsMCS。可以看到——噪声与 MCS 呈弱负相关（中位数曲线先升后缓、整体对高 MCS 不利），而平均 SINR 与 MCS 呈近单调正相关，在中高 MCS 段上升更陡、关系更稳定；低 MCS 段散点离散较大，表明相同 MCS 覆盖较宽的 SINR/噪声范围（存在调度/保守门限等非理想因素）。结论：MCS 主要受 SINR 驱动、噪声为次要因子；后续可采用分段/分箱门限或中位数回归做映射，并对极端噪声样本做稳健剪裁。

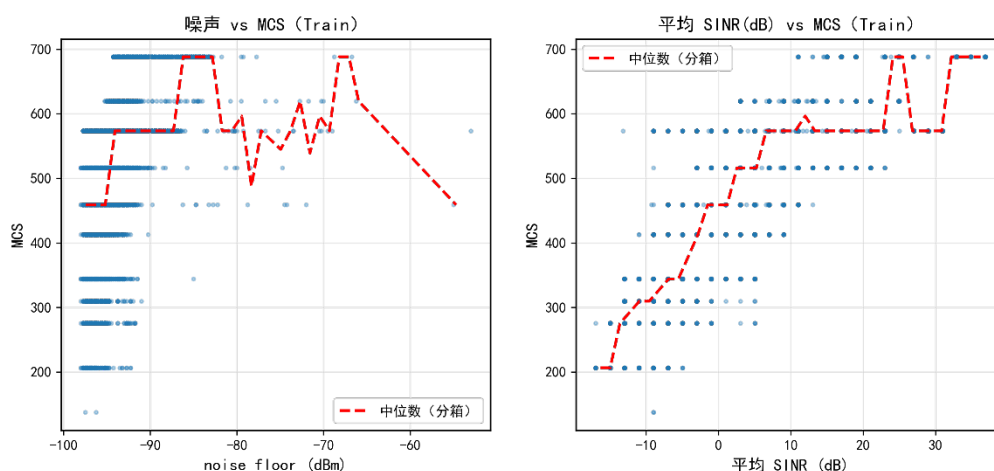


图 4-14 相关散点图

图 4-15 分别给出训练集的 MCS 分布：左图按“出现频次”降序排列并在标题中给出 Top-k 覆盖率，右图展示 11 个离散 MCS 档位的柱形计数。可以直观看到分布明显不均衡，前几类就覆盖了大部分样本，越靠后的高阶 MCS 档位样本急剧稀少（呈典型长尾）。这说明训练标签整体更偏向中低 MCS，后续做建模/评测时应考虑分层采样、类别重加权或报告分组指标，避免被头部类别主导。

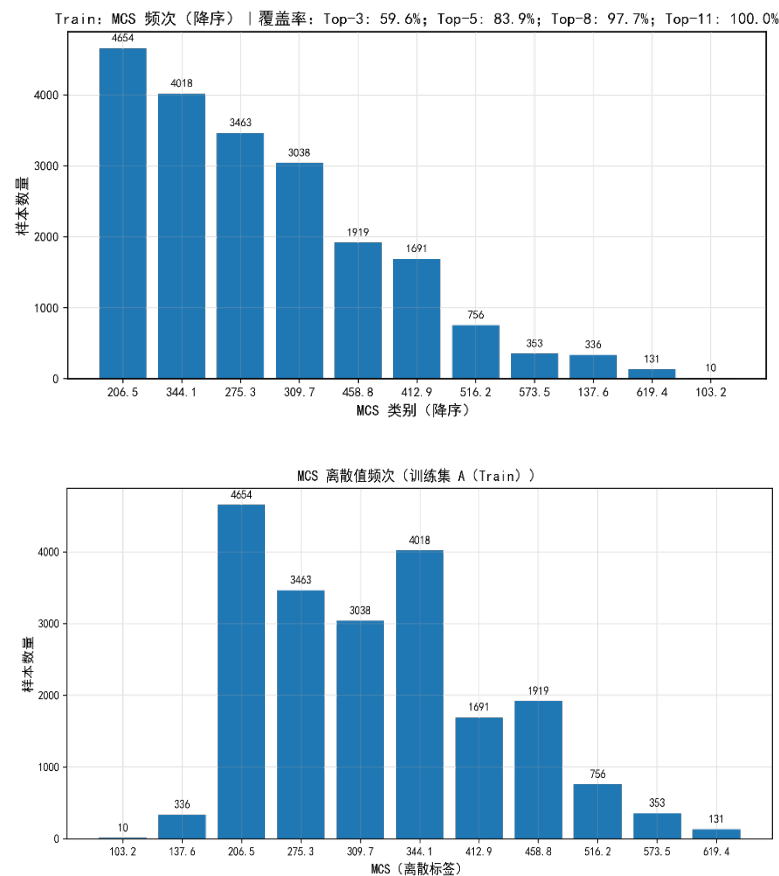


图 4-15 MCS 分布统计图

公式章 (下一章) 节 1

五、问题一的求解

5.1 问题分析

问题一旨在利用训练集 A 中完备的 MIMO-OFDM 信道信息（关闭波束赋型）及其对应的实际传输速率（MCS 标签）来对经典的 EESM 模型中的参数 α 与 β 的进行拟合，从而对不完备数据集 B 中提供的实际传输 MIMO 信道进行^[4]：

- 1) 每个载波单元的 SINR 计算；
- 2) 将所有载波单元的 SINR 进行等效 SINR 映射；
- 3) 建模信道传输速率 $rate$ ，通过传输速率索引表示。

针对问题一，整体思路流程如下图 5-1 所示。首先，在数据准备与清洗阶段，统一读取训练/验证数据并完成场景约束、时间同步与异常值剔除，保证实验数据有效性与可靠性，随后基于 CSI 计算逐载波 $SINR_i$ 并提取统计与结构特征，为下游学习器提供物理一致且分布稳定的输入；接着进入基线建模环节，以 EESM 模型为核心，通过训练集训练拟合 α, β 等参数，并采用 Isotonic Calibration 对映射进行概率与阈值校准；利用交叉验证生成 OOF（out-of-fold）预测以抑制信息泄漏^[5]，并构造可靠的基线（ISO）输出；在此基础上，继续进行增强建模，构建堆叠特征，训练梯度提升回归（HGBR）以捕捉非线性与高阶交互，并通过学习型加权对 ISO 与 HGBR 进行融合，生成兼具可解释性与表征力的预测模型；随后进入验证、推理与导出环节，严格复用训练期的变换参数，对验证集先给出 ISO 预测，再基于其派生特征获得 HGBR 与融合结果；针对序列或相邻样本的短时不一致，增设轻量的近邻平滑以降低预测抖动并提升决策稳定性；最后在评估与对比中，从训练拟合一致性、验证集性能统计等维度进行系统化审计。整体流程以 EESM 确保物理可解释的标量化抽象，以堆叠与融合提升非线性表达与泛化能力，并通过规范化验证与可视化诊断闭环，形成严谨稳健的速率预测方案。

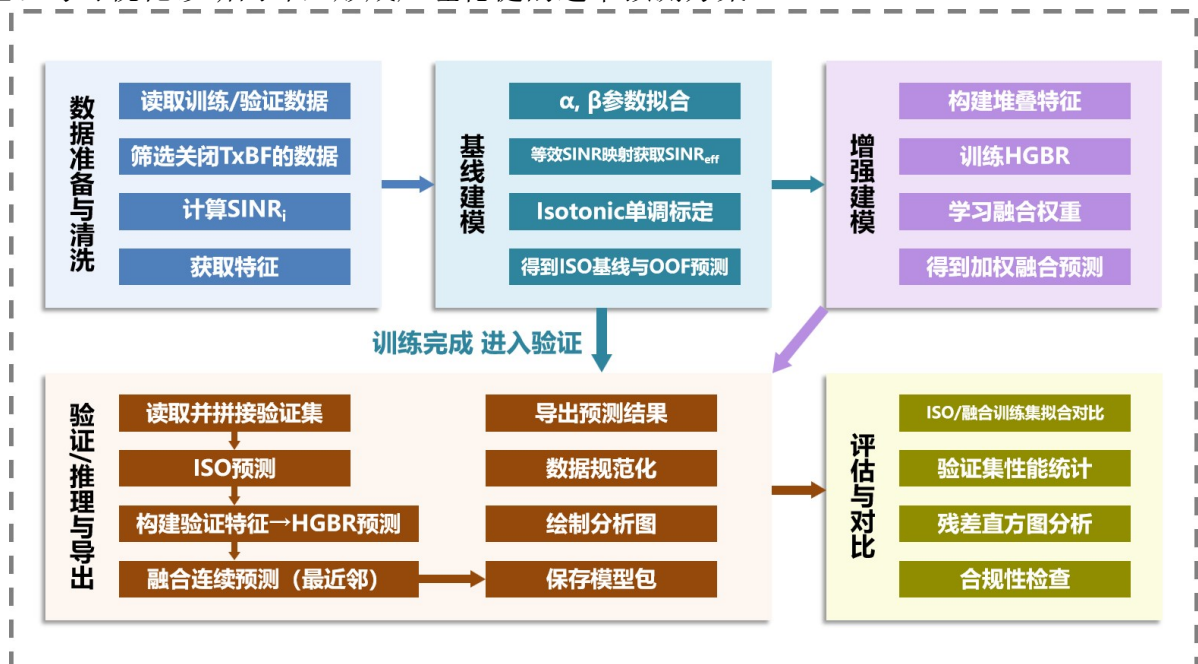


图 5-1 问题一思路流程图

5.2 数据预处理

数据预处理包括以下步骤：

1) **加载并合并数据**：读取训练集 A 下所有 Excel 文件，合并为一个 DataFrame，其中包含信道 CSI 矩阵、底噪（noise_floor）、波束赋型指示（beamforming_en）及传输速率标签（mcs）等列。验证集 B 同理加载，但不含 mcs 标签。

2) **数据清洗**：将 beamforming_en 和 noise_floor 字段转为数值型，去除无效值。对 noise_floor 若超出[-120 dBm, -50 dBm]范围则视为异常置空，并删除缺失值样本。确保仅保留波束赋型关闭（beamforming_en=0）的样本。

3) **CSI 解析**：训练表中包含 12 列 CSI 信息 csi_matrix_r0_c0...csi_matrix_r1_c3，对应 2×4×122 的复数矩阵（2 个接收天线，4 个发送天线，共 122 个子载波）。对每个样本，将这 12 列拆分合并为一个形状 2×4×122 的 MIMO 通道矩阵 $\mathbf{H}_k$ 。

4) **底噪处理**：将底噪 noise_floor 从 dBm 转换到瓦特，具体转化公式如式 5.1 所示：

$$P_{noise} = 10^{\frac{noise_floor - 30}{10}} \quad (5.1)$$

5) **计算子载波**：对于第 k 个子载波，对应的信道矩阵 $\mathbf{H}_k \in \mathbb{C}^{2 \times 4}$ ，SINR 的计算公式如式 5.2 所示：

$$SINR_k = \frac{\sum_{i,j} |\mathbf{H}_k(i,j)|^2}{P_{noise} / 122} \quad (5.2)$$

这样得到每个样本长度为 122 的 SINR 序列；同时将数值下界限控制在微小正值以避免零值下溢。上述处理完成后，可形成训练矩阵 $\mathbf{X}$ （每行 122 维 SINR 向量）、标签向量 y （对应的 MCS 索引）等，用于后续建模。

5.3 模型建立

5.3.1 指数型 ESM (EESM) 模型

等效 SINR 映射的目的是将每个载波单元 i 上的 $SINR_i$ 进行有效 $SINR_i$ 的加权处理，使其可以映射到一个合理的传输速率上^[6]。由于不能直接对 $SINR_i$ 进行加权处理，业界通常采用**等效 SINR 映射** $SINR_{eff}$ 的方式进行降维操作。在理想信道条件下，OFDM 系统中的无线信道传输速率与 $SINR_i$ 的关系可以表示成如下模型：

$$I(SINR_{eff}) = \sum_i I(SINR_i) p_i \quad (5.3)$$

其中 p_i 为信噪比 $SINR_i$ 的概率质量函数，即每个载波单元上加权系数。 $SINR_{eff}$ 可以表示为：

$$SINR_{eff} = I^{-1}(\sum_i I(SINR_i) p_i) \quad (5.4)$$

度量函数 $I(SINR_i)$ 用来表征信道传输速率，指数信息度量(Exponential information measure)

$I_{EXP}(\gamma)$ ：

$$I_{EXP}(\gamma) = 1 - \exp(-\gamma) \quad (5.5)$$

进而有：

$$\gamma_{eff} = -\ln(\sum_{i=1}^N p_i \exp(-\gamma_i)) \quad (5.6)$$

由指数信息度量函数可以获得 EESM(exponential ESM)模型。指数型 ESM 可以推广到一般情形：

$$SINR_{eff} = \alpha \Phi^{-1} \{ \frac{1}{K} \sum_{i=1}^K \Phi(\frac{SINR_i}{\beta}) \} \quad (5.7)$$

将指数型映射函数代入，最终可以获得 EESM 的形式为：

$$SINR_{eff} = \alpha (-\ln(\frac{1}{122} \sum_{i=0}^{121} e^{-\frac{SINR_i}{\beta}})) \quad (5.8)$$

其中， α 与 β 是亟待拟合的参数。

参数寻优：以训练集 A 为基准，外层对 $\alpha \in [1, 6]$ （步长 0.2）与 $\beta \in [0.1, 20]$

（logspace + 线性混合采样）做网格搜索；内层使用 **5 折/组**交叉验证。每组参数 (α, β) 的评分由“在训练折上拟合 **Isotonic** 映射、在验证折上评估平方误差”得到（见 3.2），从而直接优化“EESM→速率”的整体效果。对最优若干点再做随机邻域扰动细化，最终得到 (α^*, β^*) 。

5.3.2 Isotonic Regression 映射模型

等效 SNR 与 MCS 应保持单调不降。因此在得到 $SINR_{eff}$ 后，使用保序回归拟合从 $SINR_{eff}$ 到 MCS 的非线性映射 $\hat{f}$ ：

$$\min_{\hat{y}_i} \sum_{i=1}^n w_i (y_i - \hat{y}_i)^2 \quad \text{s.t.} \quad \hat{y}_i \leq \hat{y}_{i+1} \quad (i = 1, \dots, n-1), \quad (5.9)$$

其中 $(x_i, y_i) = (\text{SINR}_{\text{eff}}, \text{MCS}_i)$ ， w_i 为样本权重（代码中对频繁标签做了反频次加权以缓解分布不均）。模型采用 PAV 算法求得分段常数的单调函数 $\hat{f}$ ，并将输出裁剪到 $[y_{\min}, y_{\max}]$ 。

5.3.3 HGBR 回归模型

仅用 $\hat{f}(\gamma_{\text{eff}})$ 难以吸收所有频域非均匀性。为此，在 EESM+Isotonic 基线之上引入 Histogram-based Gradient Boosting Regressor(HGBR)学习剩余结构信息，特征设计：

线性域/分贝域统计：均值、标准差、极值、分位数、偏度、峰度；

阈值占比： $\Pr(\text{SINR} > 1), \Pr(\text{SINR} > 3), \Pr(\text{SINR} > 10)$ ；

频域分段 EESM：三段 $\{\text{eff_seg1}, \text{eff_seg2}, \text{eff_seg3}\}$ 及其极差、方差；

全局等效量： eff_db 、 $\phi(\text{eff_db})$ ；

噪声与来源： noise_dbm 及文件 one-hot（削弱采样批间偏移）；

单调约束：对 eff_db 特征施加正单调约束，保证等效 SINR 增大时预测速率不降低。

5.3.4 HGBR 堆叠融合 (ISO+HGBR)

最终输出为 Isotonic 与 HGBR 的线性融合：

$$\widehat{\text{MCS}} = w \cdot \widehat{\text{MCS}}_{\text{HGBR}} + (1 - w) \cdot \widehat{\text{MCS}}_{\text{ISO}}, \quad w \in [0, 1]. \quad (5.10)$$

权重 w 通过在训练集上最小化 MSE 闭式求得。

5.3.5 整体算法流程图

模型求解部分具体算法流程图如图 5-2 所示：

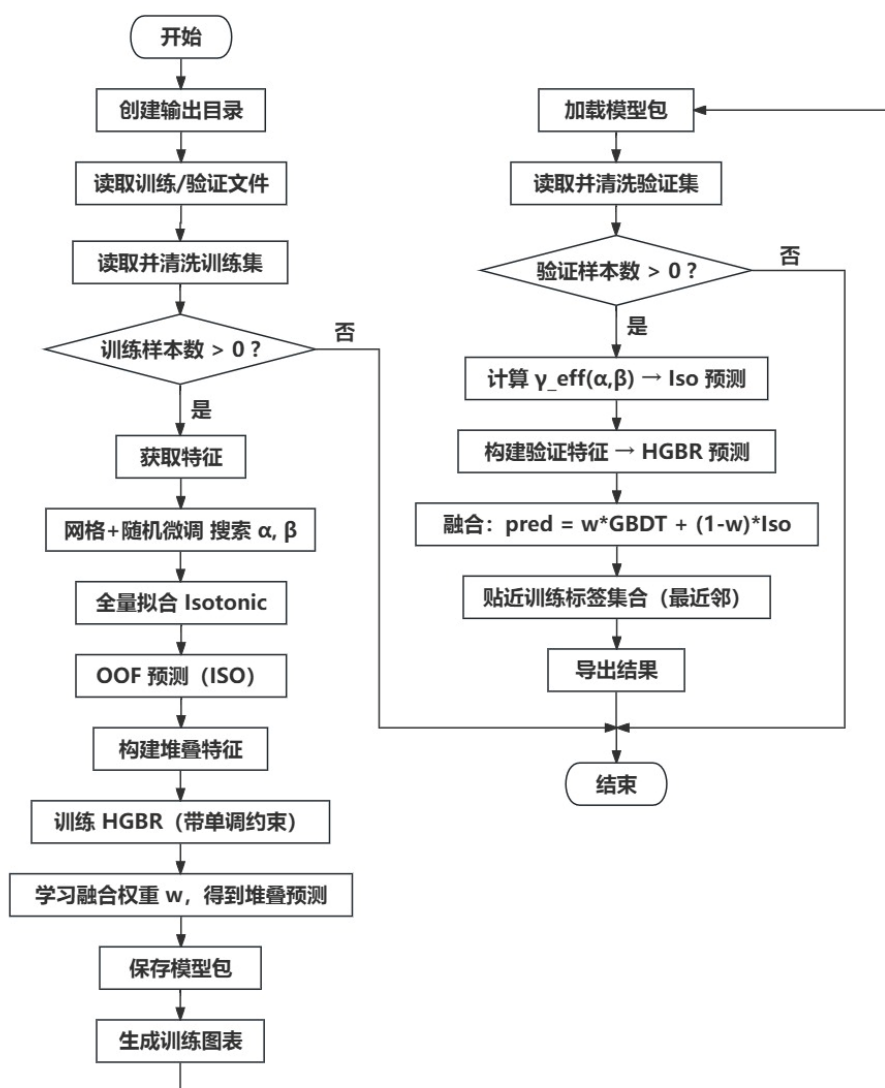


图 5-2 模型建立算法流程图

对应图 5-2 算法流程的伪代码，在表 5-1 中给出：

表 5-1 HGBR 堆叠融合（ISO+HGBR）模型伪代码

问题一伪代码

```

#初始化参数
设 NRX=2, NTX=4, K=122
设 α_grid, β_grid (粗搜), T 次局部精炼
设折数 S, 输出目录 OUT

#数据准备
读取并合并训练/验证表
数值化 beamforming_en / noise_floor / mcs
过滤 noise_floor ∉ [-120,-50] dBm
仅保留 beamforming_en==0
构造 groups = sta_mac (若无则 file_source)
  
```

```

#构造训练矩阵 X,y
for 每个训练样本 r:
    noise_per = watt(r.noise_floor) / K          # 均摊噪声
    解析 r 的 8 列 CSI → H_k, k=1..K
    for k=1..K:
        P_sig = sum(|H_k|^2)/1000
        SINR_k = P_sig / (noise_per + ε )
    X[i] = {SINR_1..SINR_K}; y[i] = r.mcs
end for

#交叉验证选择 α *, β * (粗搜 + 精炼)
best = +∞
for α ∈ α_grid:
    for β ∈ β_grid:
        做 S 折: eff_tr/eff_va = EESM_dB(X_tr/X_va; α, β)
        拟合 Iso_s: eff_tr→y_tr; 预测 ŷ_va = Iso_s(eff_va)
        计 MSE_s
        若 mean(MSE_s) 更小 → 更新 (α *, β *, best)
    end for
end for
for t=1..T: 随机扰动 (α *, β *) 并重复上步, 若更优则更新

#全量 Isotonic (固定 α *, β *)
eff = EESM_dB(X; α *, β *)
Iso = 拟合等度回归: eff→y
ŷ_iso = Iso(eff)

#OOF-ISO+元特征+HGBR+融合
iso_oof = 按 groups 做折外预测(Iso, eff, y)
F = 构造元特征(X, eff, noise_floor, file_onehot, 分段 EESM, 统计量, ...)
GBDT = 拟合(F→y); g = 预测(F)
w = clip( mean((y-iso_oof)*(g-iso_oof)) / mean((g-iso_oof)^2), 0, 1 )
ŷ_blend = w*g + (1-w)*ŷ_iso

#验证/推理/导出
对验证表:
    复用上式构造 Xv、eff_v、ŷ_iso_v
    若有 GBDT: Fv→gv, ŷ_cont = clip(w*gv + (1-w)*ŷ_iso_v, 到
[mcs_min, mcs_max])
    否则: ŷ_cont = ŷ_iso_v
ŷ_label = 将 ŷ_cont 贴近训练标签集合
导出: 总表 CSV、按文件 CSV、required.csv、规范 Excel (两列)
end

```

#结束（保存模型包与摘要）

保存 $\{\alpha^*, \beta^*, \text{Iso}, (\text{GBDT}, w), \text{label_set}, \text{特征列名}, \text{mcs_min/max}\}$ 到 OUT

5.4 模型求解

模型求解的过程主要包括四个环节：参数寻优、交叉验证、回归映射训练与堆叠融合实现。整体目标是在给定训练集数据的条件下，获得一组能够稳定映射等效信噪比 SINR_{eff} 到传输速率标签（MCS）的最优模型。具体步骤如下：

（1）参数寻优：首先对 EESM 的双参数 (α, β) 进行标定。定义损失函数为验证集上的均方误差（MSE）：

$$\mathcal{L}(\alpha, \beta) = \frac{1}{|\mathcal{D}_{\text{val}}|} \sum_{i \in \mathcal{D}_{\text{val}}} (y_i - \hat{f}_{\alpha, \beta}(x_i))^2, \lim_{\delta x \rightarrow 0} \quad (5.11)$$

其中 $\mathcal{D}_{\text{val}}$ 表示验证折样本集合， y_i 为真实 MCS 值， $\hat{f}_{\alpha, \beta}$ 表示基于参数 (α, β) 计算得到的 EESM 映射 Isotonic 回归组合模型的预测输出。

参数搜索采用两阶段方式：粗粒度网格搜索： α 在区间 $[1.0, 6.0]$ 内以 0.2 步长取值， β 在 $[0.1, 20.0]$ 内使用对数空间与等差空间混合采样。对每一组 (α, β) ，利用 K 折交叉验证计算平均 MSE 与 R^2 指标；局部扰动优化：对初步最优解 (α_0, β_0) ，在其邻域内进行随机扰动搜索，形式为：

$$\alpha = \alpha_0 + \Delta\alpha, \quad \beta = \beta_0 \cdot \Delta\beta \quad (5.12)$$

其中 $\Delta\alpha \sim \mathcal{U}(-0.3, 0.3)$ ， $\Delta\beta \sim \mathcal{U}(0.8, 1.25)$ 。最终得到一组最优参数 (α^*, β^*) 。

（2）交叉验证与稳健性检验：为防止过拟合并验证模型稳健性，采用 5 折交叉验证。即将训练集 A 划分为 5 份，依次将其中一份作为验证集，其余 4 份作为训练集，循环训练与评估。最终结果取所有验证折的平均值。通过交叉验证可以同时获得模型在不同样本分布下的泛化表现，并筛选掉在局部样本上异常优异的参数组合。

（3）Isotonic 回归映射：在确定 (α^*, β^*) 后，对所有训练样本计算其等效信噪比 SINR_{eff} ，并与真实 MCS 标签一一对应。使用 Isotonic Regression 求解以下优化问题：

$$\min_{\hat{y}_i} \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad \text{s.t. } \hat{y}_1 \leq \hat{y}_2 \leq \dots \leq \hat{y}_n. \quad (5.13)$$

最终得到一个分段常数函数 $\hat{f}(\text{SINR}_{\text{eff}})$ ，保证输出随等效 SNR 单调递增。

（4）HGBR 与堆叠融合

在 Isotonic 基线预测的基础上，构造丰富的统计与物理特征（如子载波均值、方差、阈值占比、分段 EESM 值等），使用 HGBR 模型对残差进行拟合^[7]。为了确保物理合理

性，对主要特征 SINR_{eff} 施加正单调约束。通过交叉验证选择最优超参数。最终输出通过线性融合得到：

$$\hat{y} = w \cdot \hat{y}_{\text{HGBR}} + (1 - w) \cdot \hat{y}_{\text{ISO}}, \quad w \in [0,1]. \tag{5.14}$$

其中权重 w 在训练集上通过最小化均方误差确定。

5.5 结果分析

在 `beamforming_en=0`（即 TxBF 关闭）的场景下，通过对训练集和验证集进行训练与评估，得到了 EESM 模型的待拟合参数的值于表 5-2 所示：

表 5-2 EESM 参数拟合结果（dB）

参数	参数拟合值
α	2.1999999999999997
β	0.29648733946825756

在训练集上，HGBR 堆叠融合（ISO+HGBR）堆叠模型的拟合指标为于表 5-3 所示：

表 5-3 模型拟合指标（dB）

拟合指标	参数拟合值
训练样本数（ n ）	20369
均方误差（ R^2 ）	0.828
均方误差（MSE）	1652.976
均方根误差(RMSE)	40.657
平均绝对误差（MAE）	30.952

最终使用训练好的 HGBR 堆叠融合（ISO+HGBR）堆叠模型对数据集 B 的中的三个表格预测的结果分别于表 5-4，表 5-5，表 5-6 展示（仅展示前 10 个索引）。

表 5-4 341c-f0d4-70be_0 预测结果

样本索引	速率估计结果
1	309.7
2	206.5
3	275.3
4	275.3
5	275.3
6	309.7
7	206.5
8	275.3
9	309.7
10	275.3

表 5-5 bc98-2983-907b_0 预测结果

样本索引	速率估计结果
1	206.5
2	206.5
3	206.5
4	206.5
5	275.3
6	206.5
7	275.3
8	275.3
9	206.5
10	206.5

表 5-6 d0c1-bfe4-18f0_0 预测结果

样本索引	速率估计结果
1	344.1
2	344.1
3	344.1
4	344.1
5	344.1
6	344.1
7	344.1
8	344.1
9	344.1
10	344.1

5.5.1 堆叠模型对等效 SNR - MCS 关系的精准拟合与稳健表现

在图 5-3 中我们可以看到，HGBR 堆叠融合（ISO+HGBR）模型给出的映射整体单调、平滑且物理一致，黑色渐变带基本穿过训练样本（蓝线）最密集的中位区域，说明模型抓住了主要趋势并有效抑制了局部噪声；在中等 SINR_{eff} 区间的阶梯式上升与样本跃迁位置高度吻合，映射过渡自然、无明显振荡，仅在最右端出现轻微饱和，这更像是守稳策略而非欠拟合，体现了融合后对极值段的稳健性。在极坐标视图中，橙色预测面与蓝色真实曲线在大多数扇区上同心重叠，半径变化节奏一致，显示堆叠模型在不同“角度/场景”下的泛化较好：既保持了整体幅度水平，又在局部波动处不过度追随噪声；只有少量狭窄扇区出现轻微外扩/内收，提示个别域存在弱偏差，但不影响全局贴合度，整体表现出稳健、连续且可信的映射能力。

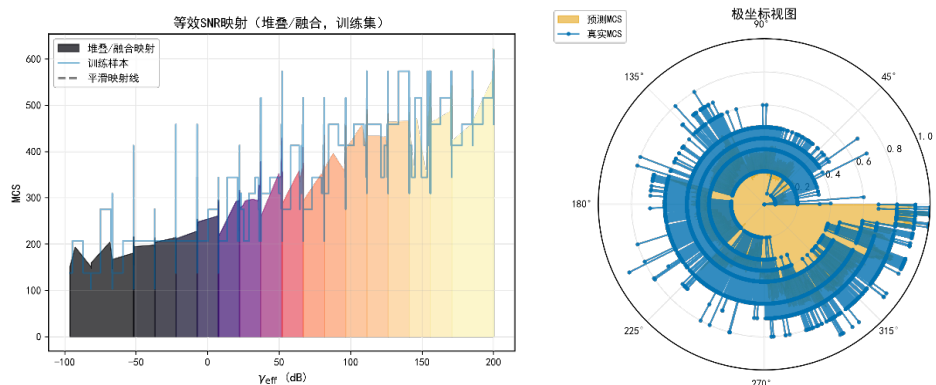


图 5-3 等效 SNR→MCS 填充面积图与极坐标视图

5.5.2 堆叠融合模型的误差结构与校准性评估

在图 5-4 中我们可以看到，热力块沿着零残差（虚线）呈狭长带状集中，说明堆叠模型在全范围内的系统性偏差很小、总体校准良好；主密度分布紧贴对角带，表示大多数

样本的误差接近 0。中高预测 MCS 区域可见轻微的暖色“尾巴”（正残差偏多），提示在高效 SNR 段模型有轻度低估真实 MCS 的倾向，但范围有限且强度不高；低 MCS 端同样未出现大面积冷色块，说明不存在明显的系统性高估。整体来看，该图体现了误差高度集中、偏差可控，堆叠模型相较仅用等值映射更稳健。各组小提琴的中位数基本贴合零线，IQR（内核厚度）随 MCS 变化保持相对窄幅，仅在中高 MCS 组略有加宽，显示误差方差轻度随难度上升而增大但仍处于可接受区间；分布形状大多近似对称，极端长尾样本比例很低，意味着模型的偶发性大误差已被有效抑制。总体特征说明：在不同预测等级上，堆叠模型的误差既无显著系统偏移，又保持较好的稳定性与一致性，能够可靠地将 γ_{eff} 到 MCS 的非线性映射落地到不同强度场景中。

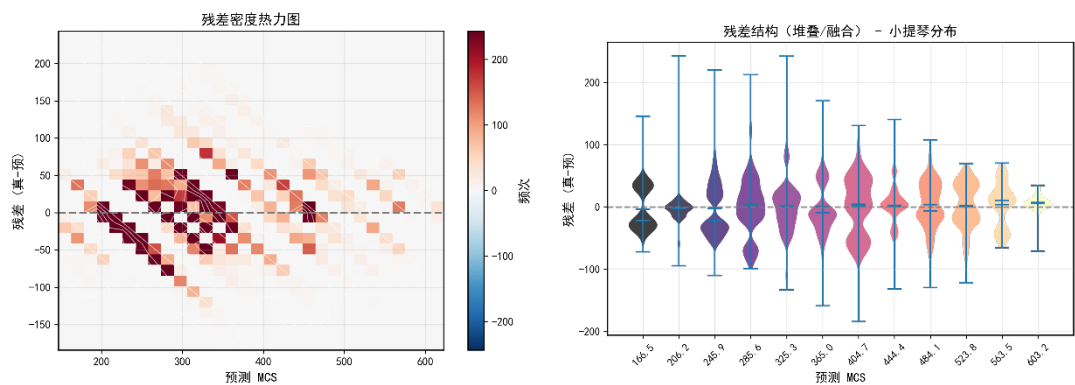


图 5-4 残差结构分析图

5.5.3 堆叠映射显著提升 MCS 拟合一致性

在图 5-5 中我们可以看到，两条密度曲线高度重叠，主峰与次峰位置基本一致，说明堆叠模型不仅学到了整体水平，也较好复刻了真实 MCS 的分布形态。预测分布在尾部更平滑，方差略小，体现了堆叠后的稳健化（对极端值不过度放大）。整体看不到系统性偏移，模型在全局层面“既准又稳”，校准良好。高密度区域沿对角线成带状分布，等密度轮廓大体平行于对角线，表明预测与真实基本同幅度变化，线性/单调映射关系被准确捕捉。散点带宽随 MCS 升高略有增大，但偏差仍集中在±小幅范围内；在高 MCS 端可见轻微保守（点群略落在对角线下方），属于合理的风险控制。总体上不存在明显的系统性错位或离群簇，堆叠模型在点对点层面的拟合一致性与稳定性均表现良好。

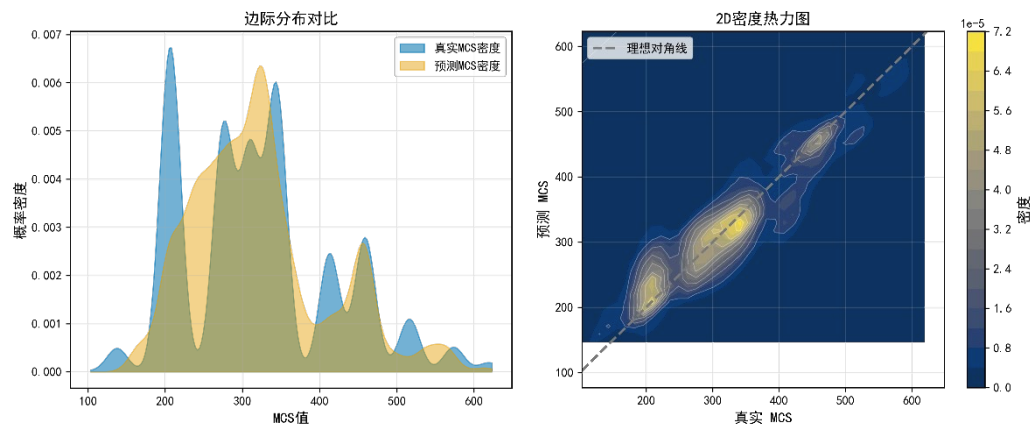


图 5-5 堆叠模型校准总览：边际密度与联合密度对比

公式章 (下一章) 节 1

六、问题二的求解

6.1 问题分析

本题同样限定在关闭发送波束赋形(TxBF)的条件下,对无线链路传输速率进行预测。但区别于问题一中基于物理机制的 EESM 近似,问题二采用基于数据驱动的机器学习方法:直接从题目中所给出的完备数据集 A 的样本的 122 个子载波的信噪比序列 (SINR) 中学习特征与传输速率的映射关系,无需显示建模底层的物理机制^[8]。问题二整体思路流程如图 6-1 所示。

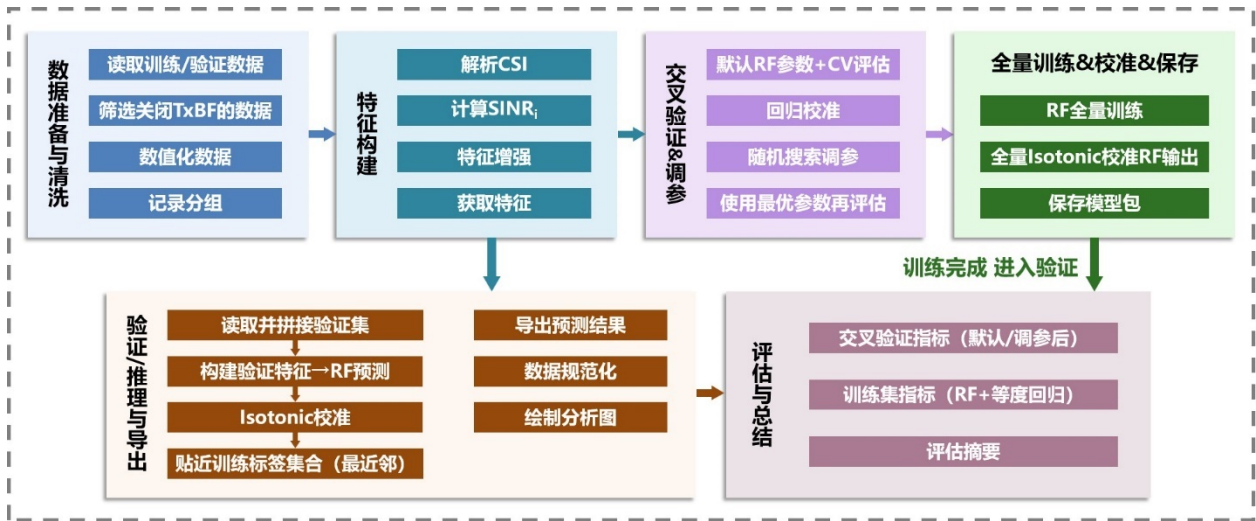


图 6-1 问题二思路流程图

建模难点：传输速率本质上是有序数值而非无序类别，虽然在本题的数据集中将传输速率视为离散速率标签，但其物理含义是连续、有强序的关系；122 维子载波之间存在复杂的非线性关系，若是使用线性方法难以充分捕捉到这种非线性关系。

模型选择：在充分考虑传输速率物理含义上的连续、强有序关系，也为了更好的捕捉到高维子载波直接的复杂非线性关系，本题选择使用随机森林（Random Forest, RF）回归模型作为非参数集成模型，通过多棵去相关的回归树进行集成平均，天然适配高维、非线性与异质噪声数据；同时模型还具备变量重要性、不确定度估计（树间标准差）与良好鲁棒性等优势，方便后续对模型性能的分析。

评估协议：为避免同一文件样本在训练和验证间互相“泄漏”，本题采用 GroupKFold 按组划分交叉验证；折数不超过独立组数量。本题的数据集在 TxBF=0 条件下有效独立组为 3，因此采用 3 折 CV。

6.2 数据预处理

数据预处理是构建可靠分析模型的关键步骤之一，原始数据的质量对于研究结果具有重要的影响。

6.2.1 基本的清洗与筛选

由于本题只考虑关闭发送波束赋形（TxBF）的样本，因此在基本的数据清洗和筛选过程中：

- 1) 将数据集表格进行读取并合并；
- 2) 在合并的表格上严格校验关键字，波束开关 *beamforming_en*、底噪信息

noise_floor、8 列 CSI 复数向量 *csi_matrix_r_i_c_j* 是否齐全，若是训练集则额外校验传输速率 *mcs*。并将波束开关 *beamforming_en* 强制为 int 类型，*noise_floor*、*mcs* 转换为 float 类型，方便后续处理；

- 3) 检查波束开关 *beamforming_en*，仅保留 *beamforming_en* = 0 的数据；

4) 基于无线链路常见噪声底范围，将 *noise_floor* 超出 [-120, -50]dBm 的观测记为异常并置为缺失，随后剔除缺失行，仅保留噪声信息可靠的样本，并重置索引。

6.2.2 信道状态信息 CSI 解析与子载波单元信噪比 SINR 计算

1) 将数据集中每个样本的 8 条链路的 CSI 序列解析为长度为 $K = 122$ 的复数向量，并自动截断/补零到 122 维。

2) 对每一行数据按子载波 k 计算等效信噪比。将功率从 mW 转化为 W ；将噪声底从 dBm 转化到 W ，并均分到每个子载波。

$$Power_k = \sum_{i=1}^2 \sum_{j=1}^4 \|H_{i,j}(k)\|^2 / 1000 \quad (6.1)$$

$$Power_{noise} = 10^{\frac{NF-30}{10}} \quad (6.2)$$

$$SINR_k = \frac{Power_k}{Power_{noise} / K + \varepsilon} \quad (6.3)$$

其中， $H_{i,j}(k)$ 为第 k 个子载波的信道增益， NF 为底噪声（ dBm ）， ε 为数值稳定项。

6.2.3 特征构建与附加统计量

1) 在筛选过后的样本集上，针对每行样本得到 122 维线性域 $SINR_k$ 序列，作为基础特征。

2) 根据所得到 122 维线性域 $SINR_k$ 序列构建一个 122 维 dB 域特征 $10\log(SINR_k)$ 。

3) 线性域的六项统计特征：线性均值、线性标准差、线性第 10 百分位、线性中位数、线性第 90 百分位，线性偏度

4) dB 均值、 dB 标准差、 dB 第 10 百分位、 dB 中位数、 dB 第 90 百分位、 dB 四分位距、 dB 偏度、 dB 线性趋势斜率、 dB 谱粗糙度

最终每个样本得到维度为 122 维线性域+122 维 dB 域+统计维度 15 的特征向量 X 。

6.3 随机森林回归模型建立

从机器学习的角度分析，传输速率预测问题本质上是一个高维回归问题。每个样本包含 122 个子载波的信噪比信息，构成 122 维的特征向量。这些特征之间可能存在复杂的交互关系和非线性组合效应，传统的线性模型往往难以准确捕捉这些关系。随机森林作为一种集成学习方法，通过构建多个决策树并进行投票，能够有效处理高维特征空间中的复杂模式。

随机森林（Random Forest，简称 RF）是一种由美国科学家 Leo Breiman 在 2001 年提出的机器学习方法。作为一种灵活且新兴的算法，随机森林在多个分类和回归问题中展现出优异的准确性，并具备广泛的应用潜力。此外，随机森林还具备内置的特征选择功能，能够有效评估各特征在特定任务中的重要性。

随机森林通过决策树算法通过递归地分割特征空间，将复杂的回归问题分解为一系列简单的决策规则。每个内部节点对应一个特征的阈值判断，叶节点对应预测值。多棵决策树的集成能够显著降低单棵树的过拟合风险，同时提高模型的泛化能力。随机森林还引入了特征随机选择和样本自助抽样机制，进一步增强了模型的鲁棒性^[9]。

在无线通信系统中，不同子载波的信道质量通常具有一定的相关性，这种相关性源于信道的物理特性和传播环境。随机森林能够自动学习这种相关性结构，并识别出对传输速率预测最重要的子载波组合。通过特征重要性分析，还可以深入理解哪些频域位置对系统性能影响最大，为系统设计和优化提供有价值的洞察。

问题二的建模目标是构建一个从多维子载波信噪比向量到传输速率的映射函数。

设训练数据集为 $\{(x^{(i)}, y^{(i)})\}_{i=1}^N$ ，其中 $x^{(i)}$ 为第 i 个样本的特征向量， $y^{(i)}$ 为其对应的传输速率标签。

随机森林模型包含 T 棵决策树，每棵树 $g_t(x)$ 在不同的训练子集上训练，并在每个分割点随机选择特征子集。第 t 棵树的训练数据通过自助抽样（Bootstrap）从原始训练集中获得，记为 $D_t = \{(x^{(i_j)}, y^{(i_j)})\}_{j=1}^{N_t}$ ，其中 $\{i_j\}$ 是从 $\{1, 2, \dots, N\}$ 有放回抽样得到的索引序列。

对于单棵决策树的构建，在每个内部节点需要选择最优的分割特征和阈值。设当前节点的样本集合为 S ，候选特征子集为 F ，其中 $|F| = \sqrt{K}$ （特征随机选择）。对于特征 $j \in F$ 和阈值 τ ，定义分割后的左右子节点样本集为：

$$S_L(j, \tau) = \{(x, y) \in S : x_j \leq \tau\} \quad (6.4)$$

$$S_R(j, \tau) = \{(x, y) \in S : x_j > \tau\} \quad (6.5)$$

分割准则采用均方误差减少量，最优分割参数为：

$$(j^*, \tau^*) = \arg \min \left[\frac{|S_L|}{|S|} \text{MSE}(S_L) + \frac{|S_R|}{|S|} \text{MSE}(S_R) \right] \quad (6.6)$$

其中, $MSE(S_L) = \frac{1}{|S|} \sum_{(x,y)} (y - \bar{y}_S)^2$, $\bar{y}_S = \frac{1}{|S|} \sum_{(x,y) \in S} y$ 为样本集 S 的平均目标值。

随机森林的最终预测通过所有决策树的平均得到:

$$\hat{y}(x) = \frac{1}{T} \sum_{t=1}^T g_t(x) \quad (6.7)$$

模型优化目标是最小化预测误差:

$$\min_{\{g_t\}_{t=1}^T} E[(y - \hat{y}(x))^2] \quad (6.8)$$

6.4 模型求解

随机森林模型的构建涉及多个超参数的选择和优化。例如, 树的数量 T 是一个重要的超参数, 过少的树可能导致欠拟合, 而过多的树虽然通常不会导致过拟合 (由于随机性的引入), 但会增加计算成本。因此超参数的选择和优化十分重要, 其直接影响到模型的性能。

由于随机搜索在高维/大候选时, 比网格搜索更高效, 因此本题中, 选择使用随机搜索和自主采样法, 设置迭代次数 $n_iter = 20$, 对树的数量 $n_estimators$, 每棵树的最大深度 max_depth , 内部节点再分裂所需的最小样本数 $min_samples_split$, 叶子节点至少要有样本数 $min_samples_leaf$, 每次分裂可见的特征子集大小 $max_features$ 进行最优参数的搜索。调参过程使用 *GroupKFold*(3) 把同一文件的样本放在同一组内, 只在组与组之间做交叉验证, 防止同一文件样本泄露。训练阶段以交叉验证的 R^2 为目标, 先用一组保守的默认值得到一套基线 CV 指标, 再采用 *RandomizedSearchCV*, 抽取 $n_iter = 20$ 组候选;

每组都做 3 折 *GroupKFold*, 评分用 R^2 。在每一折里, 先用给定超参训练 RF, 再用训练折的预测拟合一个 *Isotonic* 校准器 $\bar{y} = f_{iso}(\hat{y})$, 把它应用到验证折预测上, 此外由于训练样本标签分布不均, 因此训练时对样本使用逆频权重 (标签出现少的样本权重更高)。将搜索出的最优参数在全训练集上用逆频权重训练随机森林, 再用全训练集预测拟合一个最终 *Isotonic* 校准器并保存。

之所以要设计一个 *Isotonic* 校准器是因为, 由于样本不均衡与标签离散 (不同传输速率档位出现频次差异大), 随机森林回归的原始输出 $\hat{y}$ 往往存在系统性偏差与概率不校准: 同一预测值区间内, 真实均值与预测均值并不一致。因此我们在随机森林回归器之后串接 *Isotonic* 校准器并执行贴标签, 将校准后的连续预测 $\hat{y}$ 映射到最近的训练标签。

模型求解部分具体算法流程图如图 6-2 所示:

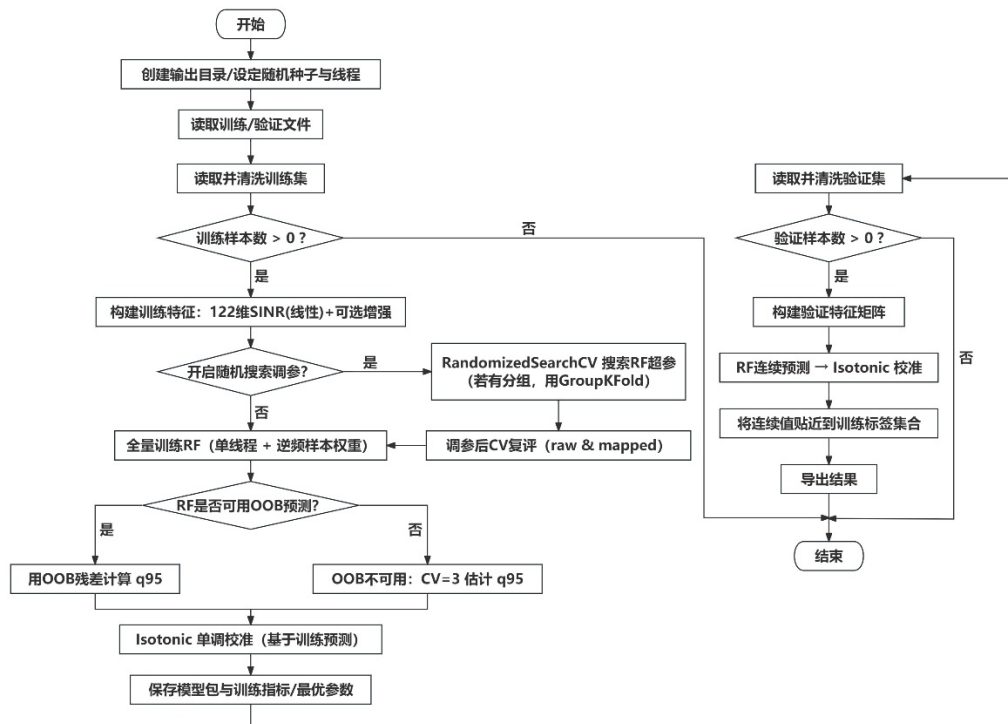


图 6-2 问题二模型建立算法流程图

对应图 6-2 算法流程的伪代码，在表 6-1 中给出：

表 6-1 问题二伪代码

问题二伪代码

```

#初始化参数
K=122; USE_AUG ∈ {T/F}; S 折; RF 默认/调参 n_iter          #K 为子载波数, USE_AUG 表示是否启用特征增强

#数据准备
读取训练/验证集; 数值化列; 过滤 noise_floor ∉ [-120,-50]
仅取 beamforming_en==0; groups=sta_mac 或 file_source 的数据

#训练特征
X,y,groups ← 空
for r ∈ TRAIN:
    noise_per = watt(r.noise_floor)/K
    seq = Σ |CSI|^2/1000 ÷ (noise_per + ε)  # 122 维 SINR
    feat = (USE_AUG)? [seq, 10log10(seq), 15 统计] : seq
    追加 X ← feat, y ← r.mcs
end
label_set = 唯一(y)升序

#CV 与调参
cv_default = S 折评估(RF 默认, 可折内 Iso)
若调参: best_params = 随机搜索(RF,n_iter,评分 R^2)

```

```

cv_tuned = S 折评估(RF=best_params)

#全量训练与校准
RF ← 拟合(X,y, 逆频权重)
iso ← 拟合等度回归(RF(X)→y)
保存{RF, iso, label_set, mcs_min/max}

#验证/推理/导出
构建 VALID 特征 Xv(与训练同口径)
y_cont = iso( RF(Xv) )
y_label = 贴近 y_cont → label_set
导出: pred_valid.csv、by_file/*、q2_required_outputs.csv、问题 2-nontxbf-
<mac>.xlsx

#总结
写 summary_q2_rf.json (样本量、CV、Train、参数)

```

6.5 模型结果与分析

经过搜索后得到的随机森林最优参数如表 6-2 所示。

表 6-2 随机森林最优参数

参数名称	参数值
<i>n_estimators</i>	500
<i>min_samples_split</i>	10
<i>min_samples_leaf</i>	16
<i>max_features</i>	0.6
<i>max_depth</i>	12

使用搜索得到的最优参数，对随机森林模型在关闭 TxBF 条件下的完备数据集 A 上进行全量拟合，得到最终的模型评价指标如表 6-3 所示。

表 6-3 最优参数下的随机森林评价指标

评价指标	值
R^2	0.765
<i>RMSE</i>	47.557
<i>MAE</i>	36.846

根据表格中的性能指标可知，模型中还存在 23.5% 的方差未被解释，可能是由于数据预处理时所构建的信息不够充分，漏掉了一些重要的特征，如信道矩阵秩、条件数、最大奇异值等，导致随机森林模型没有抓住信道信息中存在的非线性关系和互相作用。

图 6-3 为随机森林特征重要性的茎叶图，选择其中最重要的 20 个特征进行绘图，如图 6-4 所示，可知道其中最重要的特征为第 87 子载体的 *dB* 域特征，其次则为第 75 子载体的线性域特征，最重要的 20 个特征基本上是关键子载波段的线性域特征和 *dB* 域特征。

整体上来看模型同时依赖全局统计特征与物理先验相关特征（关键子载波段），贡献分散，无单一特征“垄断”性能，指向较好的鲁棒性与可解释性。

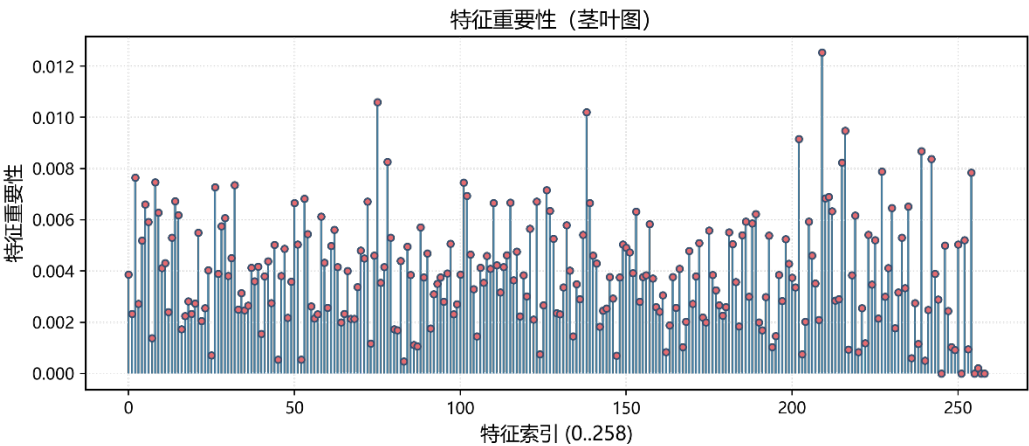


图 6-3 特征重要性茎叶图

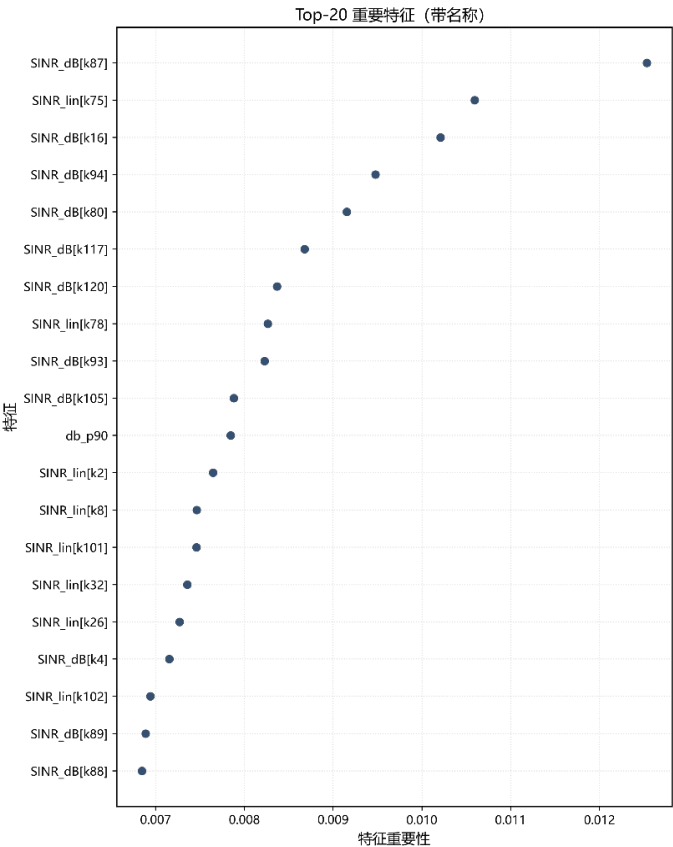


图 6-4 Top-20 特征重要性

随机森林回归的原始输出经过 Isotonic 校准器再映射到最近的训练标签后，准确率为该模型对 mac 地址为 341c-f0d4-70be 的表中样本索引标记为 1-10 的预测结果如表 6-4 所示。

表 6-4 mac 地址为 341c-f0d4-70be 的预测结果	
样本索引	速率估计结果
1	309.7
2	206.5
3	275.3
4	275.3
5	275.3
6	275.3
7	275.3
8	275.3
9	275.3
10	275.3

该模型对 mac 地址为 bc98-2983-907b 的表中样本索引标记为 1-10 的预测结果如表 6-5 所示。

表 6-5 mac 地址为 bc98-2983-907b 的预测结果	
样本索引	速率估计结果
1	206.5
2	206.5
3	206.5
4	206.5
5	275.3
6	275.3
7	275.3
8	275.3
9	206.5
10	206.5

该模型对 mac 地址为 d0c1-bfe4-18f0 的表中样本索引标记为 1-10 的预测结果如表 6-6 所示。

表 6-6 mac 地址为 d0c1-bfe4-18f0 的预测结果	
样本索引	速率估计结果
1	344.1
2	344.1

3	344.1
4	344.1
5	344.1
6	344.1
7	344.1
8	344.1
9	344.1
10	344.1

公式章 (下一章) 节 1

七、问题三的求解

7.1 问题分析

本题限定在开启发送波束赋形 ($T_xBF=1$) 条件下对无线链路传输速率进行估计。波束赋形技术通过在发射端对信号进行预编码, 能够将发射功率集中在信道条件较好的方向上, 从而显著提高接收信噪比和系统性能。这种技术在现代无线通信系统中得到广泛应用, 特别是在 MIMO 系统中能够充分发挥空间分集和复用的优势。

从数学角度分析, 波束赋形的核心是设计最优的发射权值向量, 使得接收端的信号质量最大化。在已知信道状态信息的前提下, 最优波束赋形问题可以表述为一个约束优化问题。通过奇异值分解 (SVD) 技术, 可以将信道矩阵分解为正交矩阵和对角矩阵的乘积, 从而直接得到最优的波束赋形向量。

SVD 方法的理论基础源于矩阵理论中的主成分分析思想。信道矩阵的奇异值反映了不同空间模式的增益大小, 最大奇异值对应的特征向量指向了信道增益最大的方向。通过将发射功率集中在这个方向上, 可以最大化接收信号的功率, 从而提高信噪比和传输性能。

与无波束赋形场景相比, 开启波束赋形后的信道特性发生了根本性变化。原本的 2×4 MIMO 信道被等效为一个增强的单输入多输出 (SIMO) 信道, 其中发射端的多个天线被视为一个等效的发射源, 接收端仍然保持多个接收天线。这种等效变换简化了后续的信号处理和性能分析, 同时保持了数学模型的精确性。

因此与问题二“纯数据驱动”的思路不同, 问题三采用物理模型+数据驱动的混合范式: 先由计算开启 T_xBF 情况下的每个子载波的 $SINR_k$ 序列, 再通过 EESM 压缩为等效 $SINR_{eff}$, 并以 Isotonic 学习单调映射; 再在此基础上提取相关的统计特征与 SVD 结构特征, 用来训练 HistGradientBoosting Regressor (HGBR) 回归器^[10]; 最后将 HistGradientBoosting Regressor (HGBR) 回归器与物理模型做岭化线性融合, 最后用 OOF-Isotonic 进行二次校准。

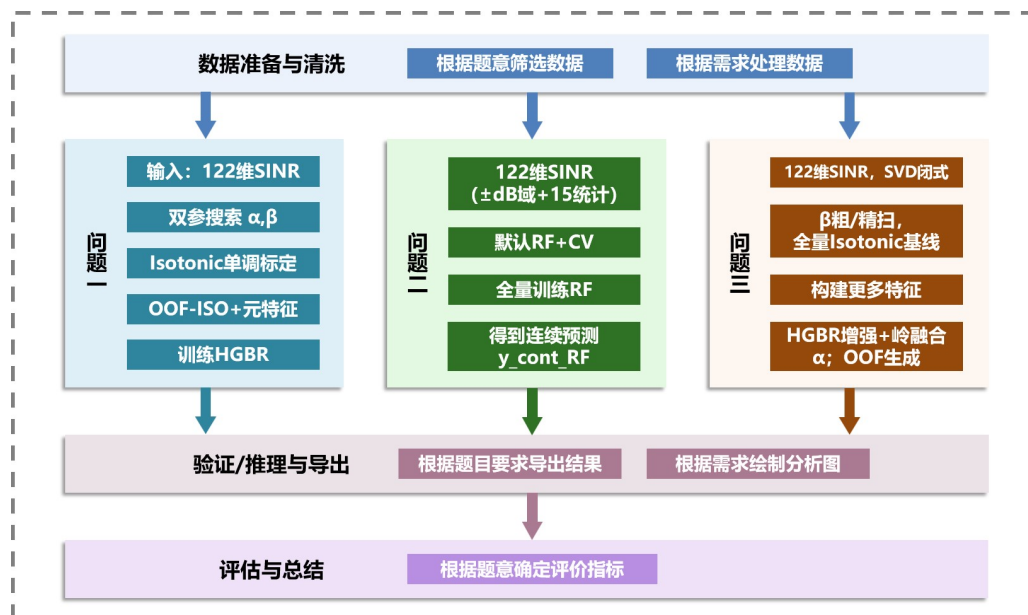


图 7-1 问题三思路流程图

建模难点: MCS 在物理上连续且有序, 但标注为离散档位, 直接回归容易受标签不均

衡与极端样本影响；开启 TxBF 条件下，CSI 的两路接收与四路发射耦合，子载波间存在复杂的非线性与相关性。

模型选择：物理模型选择使用 EESM 结合 Isotonic，具有较强的可解释性；数据通道选择使用 HGBR，来捕捉到信道状态信息中的高位非线性；将两个模型通过闭式岭化 α 融合起来，再通过 OOF-Isotonic 二次校准。

评估协议：同问题 2，使用 GroupKFold 防止同一文件在训练和验证间互相“泄漏”，本题的训练文件有六个有效独立组，因此可采用 5 折 CV。

7.2 数据预处理

基本的清洗与筛选

由于本题只考虑开启发送波束赋形（TxBF）的样本，因此在基本的数据清洗和筛选过程中

- 1) 将数据集表格进行读取并合并；
- 2) 在合并的表格上严格校验关键字，波束开关 *beamforming_en*、底噪信息

noise_floor、8 列 CSI 复数向量 *csi_matrix_r_i_c_j* 是否齐全，若是训练集则额外校验传输速率 *mcs*。并将波束开关 *beamforming_en* 强制为 int 类型，*noise_floor*、*mcs* 转换为 float 类型，方便后续处理；

- 3) 检查波束开关 *beamforming_en*，仅保留 *beamforming_en* = 1 的数据；

4) 基于无线链路常见噪声底范围，将 *noise_floor* 超出 $[-120, -50]$ dBm 的观测记为异常并置为缺失，随后剔除缺失行，仅保留噪声信息可靠的样本，并重置索引。

7.2.1 信道状态信息 CSI 解析与子载波单元信噪比 SINR 计算

1) 将数据集中每个样本的 8 条链路的 CSI 序列解析为长度为 $K = 122$ 的复数向量，并自动截断/补零到 122 维。

2) 对每一行数据按子载波 k 计算等效信噪比。将功率从 mW 转化为 W ；将噪声底从 dBm 转化到 W ，并均分到每个子载波，具体公式同问题二。

7.2.2 等效 SNR 与特征构建

通过 EESM 的公式对等效 $SINR_{eff}$ 进行计算。

$$SINR_{eff} = -\beta \ln\left(\frac{1}{K} \sum_{k=1}^K e^{-(SINR_k/\beta)}\right) \quad (7.1)$$

7.2.3 特征构建

1) 对 122 维子载波的线性域和 dB 域进行统计特征：均值、标准差、P10/中位/P90、偏度、IQR、线性趋势斜率、谱粗糙度等，同问题 2。

2) SVD 结构特征：最大奇异值的平均 $\bar{s}_1$ 、次大奇异值的平均 $\bar{s}_2$ 、平均条件数

$cond_{mean}$ 、平均成形增益 $\bar{\lambda}_{max}$ 。由于问题三的模式构建在数学上更加复杂，因为需要处理复数矩阵的奇异值分解和相关的数值计算问题，SVD 分解的计算复杂度为 $O(\min(mn^2, m^2n))$ 。信道本身是两根接收天线和四根发射天线，但在开启 TxBF 的情况下，最优接收 SNR 等于信道矩阵的最大奇异值的平方，而最大奇异值的平方既可以从 $H_k^T H_k (4 \times 4)$ 取，也可以从 $H_k^T H_k (2 \times 2)$ 取，因为两者的非零特征值完全相同。因此，为例高效，本文在理论上因此等价于逐子载波 SVD 的基础上，针对 2×2 接收端结构推导了闭式最大特征值的向量化公式以替代逐子载波 SVD，提高了效率且保持与最大奇异值决定 SNR 的一致性。

对每个子载波，令 h_{1k} ， h_{2k} 为 H_k 的两行（两路接收），则有：

$$a_k = \|h_{1k}\|_2^2, d_k = \|h_{2k}\|_2^2, b_k = h_{1k} \bar{h}_{2k}^T \quad (7.2)$$

Gram 矩阵有：

$$G_k = \begin{bmatrix} a_k & b_k \\ \bar{b}_k & d_k \end{bmatrix} \quad (7.3)$$

该 2×2 Gram 矩阵有两个特征值，即 H_k 的奇异值平方有：

$$\begin{cases} \lambda_{\max,k} = \frac{1}{2}[(a_k + d_k) + \sqrt{(a_k - d_k)^2 + 4|b_k|^2}] \\ \lambda_{\min,k} = \frac{1}{2}[(a_k + d_k) - \sqrt{(a_k - d_k)^2 + 4|b_k|^2}] \end{cases} \quad (7.4)$$

于是：

$$s_{1,k} = \sqrt{\lambda_{\max,k}}, s_{2,k} = \sqrt{\lambda_{\min,k}} \quad (7.5)$$

取：

$$s_1 = \max s_{1,k}, s_2 = \max s_{2,k}, cond = \frac{s_1}{s_2}, gain = \lambda_{\max} \quad (7.6)$$

分别代表最大奇异值，次大奇异值，条件数，成型增益。

在对 122 个子载波求上述四个特征的平均值即可获得 SVD 的特征，最大奇异值的平均、次大奇异值的平均、平均条件数、平均成形增益。

1) 多个 β 的 EESM 向量和最优 β^* 的等效 SINR 显示特征；

2) 数据集底噪信息

7.3 融合模型建立于求解

7.3.1 物理模型

首先在物理先验框架下给出一个可解释的单变量基线。根据 $SINR_{eff}$ 的公式可知，需要调整 β 来找到一个由 $SINR_{eff} \mapsto mcs$ （传输速率）的单调映射关系。

EESM 参数的优化过程需要考虑波束赋形对信道统计特性的影响。与无波束赋形场景相比，开启波束赋形后的信噪比分布通常具有更大的动态范围和不同的统计特性。这可能影响最优 β 参数的选择，需要在更大的参数空间内进行搜索。因此，为确定最优 β^* ，本文对 β 做两阶段扫描，先确定一个较大的区间并以一个较大的步长进行初步的计算，在寻得一个相对较优的区间后在其以一个较小的步长做精扫。对每个候选 β ，用 GroupKFold 在训练集上进行交叉验证，将 $SINR_{eff}(\beta)$ 作为自变量，对目标速率拟合 Isotonic 回归，得到映射 $g_\beta : SINR_{eff} \mapsto \hat{y}$ 。以验证折的 MSE 作为指标选择：

$$\beta^* = \arg \min_{\beta} MSE_{CV}(y, g_\beta(SINR_{eff}(\beta))) \quad (7.7)$$

最终在全量训练集上以 β^* 重训单调映射 $g_{iso} = g_{\beta^*}$ ，得到物理基线预测

$$\hat{y}_{iso} = g_{iso}(SINR_{eff}(\beta^*)) \quad (7.8)$$

并将连续输出裁剪到训练标签的范围内，以便后续贴最近标签用于离散评估。

7.3.2 ML 模型

为更充分地刻画 122 个子载波的结构信息，在 TxBF 开启的情况下，本文构造多源、低维且稳定的特征，并以直方梯度提升回归（HistGradientBoosting Regressor, HGBR）进行学习，进一步学习残差结构与非线性规律，提升连续空间拟合与离散贴档的稳定性与精度。输入特征（在开启 TxBF 的情况下）设计：

与问题二类似，可获得第 k 个子载波的线性域，和其 dB 域。

1) 统计特征（对 122 维的线性域与 dB 域进行特征提取）：均值、标准差、P10/中位/P90、偏度、IQR、线性趋势斜率（沿子载波索引拟合的一次项）、谱粗糙度（dB 序列一阶差分的标准差）等，具体问题 2 的统计特征；

2) SVD 结构特征见前文；

3) EESM 多 β 向量特征，取一组 β （本文取 9 个 5.0, 7.5, 10.0, 12.5, 15.0, 17.5, 20.0, 22.5, 25.0）计算一组物理先验 SINR 等效特征，并额外加入以交叉验证筛选出的 β^* 计算的 SINR 等效特征；

4) 数据集中的底噪信息。

上述特征维度合计统计特征 15 维+SVD 特征 4 维+EESM 多 β 向量特征 10 维+数据集底噪信息 1 维，共计 30 维。

采用 HistGradientBoostingRegressor，并针对噪声与离群点选择鲁棒超参，本文选用的 HGBR 配置如表 7-1：

表 7-1 HGBR 相关配置

参数名称	参数值
max_leaf_nodes	63
$min_samples_leaf$	60
$l2_regularization$	0.01
$learning_rate$	0.03
max_iter	900

同时为缓解训练的速率标签不均匀，对训练样本按等频 10 分箱计算逆频权重并归一化为均值 1，作为样本权重，用以缓解某些 MCS 档位样本稀缺导致的偏置。并使用 GroupKFold 交叉验证，避免同域样本泄露。最终模型输出维连续预测的 $\hat{y}_{ml}$ 。

7.3.3 融合模型与二次校准

为兼顾物理模型的可解释性与数据驱动的非线性特征捕捉能力，我们在 $\hat{y}_{iso}$ 和 $\hat{y}_{ml}$ 之间进行凸融合，随后以 OOF-Isotonic 校准修正全局偏差。

1) 岭化闭合融合：设融合权重 $\alpha \in [0, 1]$ ，融合预测 $\hat{y}_{blend} = \alpha \hat{y}_{iso} + (1 - \alpha) \hat{y}_{ml}$ 。最小化带岭正则二范数风险：

$$\min_{\alpha} \|y - [\alpha \hat{y}_{iso} + (1 - \alpha) \hat{y}_{ml}]\|_2^2 + \lambda \alpha^2 \quad (7.9)$$

令 $r = \hat{y}_{iso} - \hat{y}_{ml}$ ，则有

$$\alpha^* = clip_{[0,1]} \left(\frac{(y - \hat{y}_{ml})^T r}{\|r\|_2^2 + \lambda} \right), \quad \lambda = 10^{-3} \quad (7.10)$$

2) OOF-Isotonic 二次校准：

考虑到标签离散与分布偏斜，回归器的连续输出在不同区间常存在系统性偏差。为避免信息泄漏，我们在交叉验证的外侧折（OOF）上拟合二次单调校准器：

用 GroupKFold 产生折，逐折训练 HGB/融合并记录验证折的 OOF 预测 $\tilde{y}_{ml}$ 和 $\tilde{y}_{blend}$ ：

以 $(\tilde{y}_{ml}, y)$ 拟合映射 g_{ml} ，以 $(\tilde{y}_{blend}, y)$ 拟合映射 g_{ml} ；

在得到 $\hat{y}_{iso}$ 、 $\hat{y}_{ml}$ 和 $\hat{y}_{blend}$ 之后，再应用后置校准：

$$\hat{y}_{final} = \begin{cases} g_{blend}(\hat{y}_{blend}), & \text{使用岭化闭合融合} \\ g_{ml}(\hat{y}_{ml}), & \text{不使用岭化闭合融合} \end{cases} \quad (7.11)$$

7.4 模型结果与分析

通过开启 TxBF 的完备数据集 A 对融合模型进行训练，得到的三种模型性能指标如表 7-2 所示。

表 7-2 三种模型性能指标（组内交叉验证时）

评价指标	物理基线模型	ML 基线模型	融合模型
R^2	0.628	0.702	0.702
$RMSE$	62.195	55.689	55.689
MAE	50.231	43.521	43.521
$Top-1$	0.288	0.391	0.391
± 1	0.810	0.861	0.861

可以发现相较于物理基线模型，ML 基线模型与融合模型有一个较大的改进幅度，ML 基线模型的 R^2 比物理基线模型高了 0.074，而 $RMSE$ 和 MAE 则分别低了 6.506 和 6.71，精确命中率。可见仅通过物理先验和单调映射的模型无法覆盖全部链路的结构信息。ML 基线模型所构建的多维特征组合能够有效补充物理基线模型未能刻画的非线性/结构信息。

在本题中额外引入两个新的指标 $Top-1$ 和 ± 1 ，因为本题的速率档位本质上是离散且有序的目标，而模型输出的是连续值，这两个指标能把连续预测转成是否选对和是否在可接受范围内的评价，更贴合通信系统选档的实际决策。其中 $Top-1$ 表示模型预测值经离散化映射后选中的速率档是否等于真实速率档（真实命中率），它直接衡量能否一次选对。 ± 1 表示模型预测值经离散化后选中的速率档与真实速率档是否成立（容错命中率）。工程中相邻档位的调制/编码差异较小，常能在误差和测量噪声下仍维持可接受的吞吐，因此相差一档通常仍可用超过一档才可能带来明显的掉速或误块率上升。该指标因此更符合“可用性”的容错评估。根据表 7-2 可知 ML 基线模型和融合模型这两个指标均好于物理基线模型，ML 基线模型和融合模型 $Top-1$ 比物理基线模型的 $Top-1$ 高了 0.103，而 ± 1 则高了 0.051。

此外，ML 基线模型的评价指标与融合模型的评价指标完全一致，是因为在岭回归闭式当中求得的 $\alpha^* = 0$ ，这意味着 ML 以及吸收了 EESM 的信息，在 ML 模型构建特征时，以及将 $SINR_{eff}(\beta^*)$ 作为特征输入到 ML 模型当中了，再融入物理基线模型的话反而不会带来额外的收益，因此融合模型变退化为了 ML 模型，是预期的现象。

图 7-2 为 β 参数扫描图，其扫描的面积阶梯图清晰展示了 EESM 参数优化的全过程。CV-MSE 曲线呈现明显的凸函数特征，在 $\beta = 17.4$ dB 出达到最小值。可见开启波束赋形后，不同子载波的信噪比差异更加明显，需要更大的 β 值来平衡各子载波的贡献。填充面积的可视化突出了最优参数附近的平坦区域，表明参数选择具有一定的鲁棒性。最优点的散点标记和垂直虚线清晰指示了选择的 β 值，为后续的等效 SNR 计算提供了准确的参数基础。

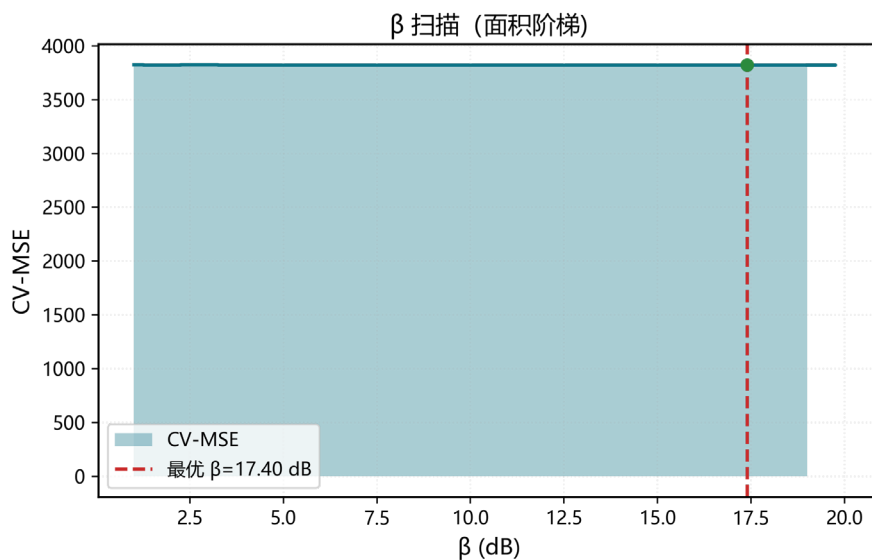


图 7-2 β 参数扫描图

图 7-3 为该融合模型的离散化混淆矩阵，可见该混淆矩阵是典型的对角线占有结构，主要误差都落在 ± 1 档。多数标签的对角元素位于 0.8 附近，个别中档位较低，同时向相邻档位外溢，说明这些档位边界模糊、更易被上档吸走，若要进一步提高性能指标则需要设法明确这些边界。

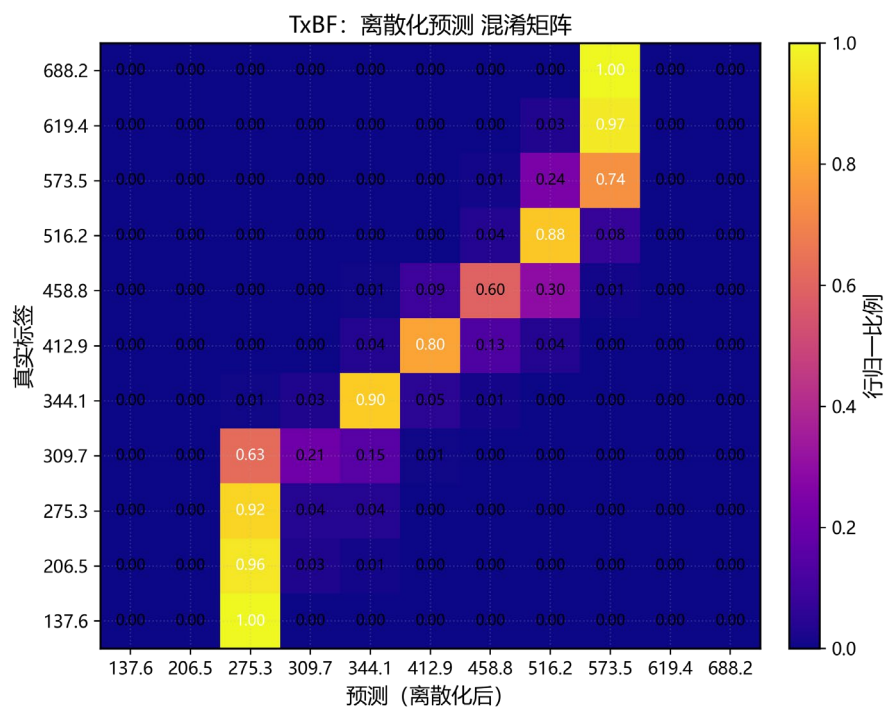


图 7-3 融合模型离散化混淆矩阵

图 7-4 为 ML 模型的前 20 特征重要性，从图中可见呈现一强多弱的情况，第一重要性的是 $\bar{s}_2$ ，推测可能是相关特征的信用分配和数据里它恰好最能纠正 EESM 的剩余误差。

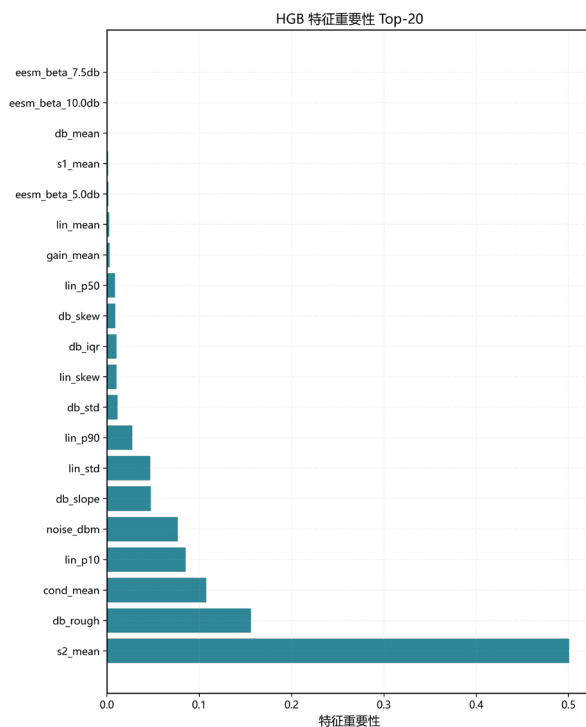


图 7-4 ML 基线模型的前 20 特征重要性

图 7-5 是 SVD 相关性热图，SVD 统计量与 MCS 的相关性热图揭示了波束赋形效果的机理。最大奇异值与 MCS 的强正相关表明主信道模式的增益直接影响传输性能，这验证了 SVD 波束赋形的理论基础。条件数与 MCS 的相关性反映了信道空间特性对性能的影响，

高条件数通常对应更好的波束赋形效果。平均增益与 MCS 的相关程度量化了波束赋形技术的直接效益。相关性矩阵的色彩编码便于识别强相关和弱相关关系，为理解波束赋形机理提供了定量工具。

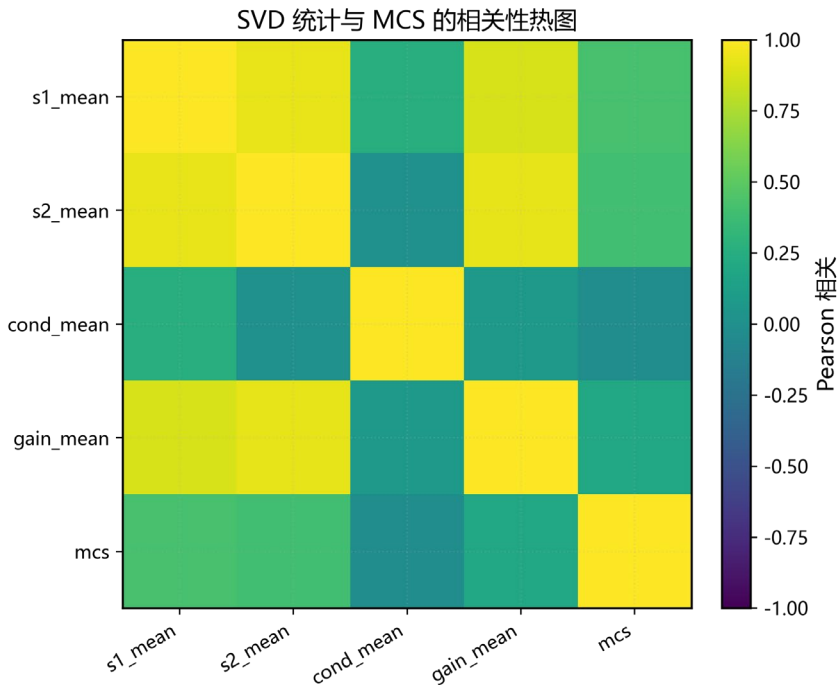


图 7-5 SVD 相关性热图

该模型对 mac 地址为 341c-f0d4-70be 的表中样本索引标记为 1-10 的预测结果如表 7-3 所示

表 7-3 mac 地址为 341c-f0d4-70be 的预测结果	
样本索引	速率估计结果
1	412.9
2	412.9
3	412.9
4	412.9
5	412.9
6	412.9
7	412.9
8	412.9
9	412.9
10	344.1

该模型对 mac 地址为 508e-4963-d5dc 的表中样本索引标记为 1-10 的预测结果如表 7-4 所示。

表 7-4 mac 地址为 508e-4963-d5dc 的预测结果	
样本索引	速率估计结果

1	516.2
2	516.2
3	516.2
4	516.2
5	516.2
6	516.2
7	516.2
8	516.2
9	516.2
10	516.2

该模型对 mac 地址为 508e-4963-e122 的表中样本索引标记为 1-10 的预测结果如表 7-5 所示。

表 7-5 mac 地址为 508e-4963-e122 的预测结果

样本索引	速率估计结果
1	573.5
2	516.2
3	573.5
4	516.2
5	516.2
6	573.5
7	573.5
8	573.5
9	573.5
10	516.2

该模型对 mac 地址为 bc98-2983-907b 的表中样本索引标记为 1-10 的预测结果如表 7-6 所示。

表 7-6 mac 地址为 bc98-2983-907b 的预测结果

样本索引	速率估计结果
1	412.9
2	412.9
3	412.9
4	412.9
5	412.9
6	412.9

7	412.9
8	412.9
9	412.9
10	412.9

该模型对 mac 地址为 d0c1-bfe4-18f0 的表中样本索引标记为 1-10 的预测结果如表 7-7 所示。

表 7-7 mac 地址为 d0c1-bfe4-18f0 的预测结果	
样本索引	速率估计结果
1	573.5
2	573.5
3	516.2
4	516.2
5	516.2
6	516.2
7	573.5
8	516.2
9	516.2
10	516.2

该模型对 mac 地址为 fc02-9668-e2fd 的表中样本索引标记为 1-10 的预测结果如表 7-8 所示

表 7-8 mac 地址为 fc02-9668-e2fd 的预测结果	
样本索引	速率估计结果
1	516.2
2	516.2
3	516.2
4	516.2
5	516.2
6	516.2
7	516.2
8	516.2
9	516.2
10	516.2

八、模型评价与改进

8.1 模型优点

- 1) **物理可解释+工程通用**: 把 122 维子载波 SINR 先做 EESM (α 、 β) 得到等效 SNR, 再映射到 MCS。EESM/ESM 属于 3GPP/系统级仿真里最常用的 PHY 抽象, 计算轻、能跨场景迁移, 且与“按目标 BLER 选调制编码”的流程一致;
- 2) **单调校准, 稳健对齐标签**: 等效 SNR→MCS 用 IsotonicRegression, 天然保证**单调性**, 能在存在分布偏移或噪声的情况下把回归值“拉回”到合理区间, 降低系统性偏差;
- 3) **弱假设+强表达**: 在 EESM 的可解释支撑下, 再用 HistGradientBoostingRegressor (带**单调约束**) 做轻量堆叠, 将“非线性残差”补齐, 同时约束主特征 (eff_db) 与输出**单调**, 减少过拟合风险;
- 4) **数据效率好**: 相比端到端深网, EESM+Isotonic 的参数很少、样本需求低; HGBR 在中等数据量下也能稳定收敛。

8.2 模型缺点

- 1) **EESM 的普适性与可辨识性**: β 往往与调制/码率相关, 很多文献建议分 MCS 标定或换用 MIESM (基于互信息) 以提升跨信道泛化; 单一 (α, β) 在不同衰落与编码组合下可能出现系统性偏差;
- 2) **尾部/极端场景**: Isotonic 是分段常数/线性阶梯, **外推能力弱**; 你用 winsor 限幅能稳住训练, 但会牺牲极端低/高 SNR 区的精度;
- 3) **标签量化与异方差**: 最终“贴近标签集”的**就近取整**会引入量化误差; 同时 MCS 残差往往随 γ_{eff} 增大而异方差, 普通平方损失对厚尾不鲁棒;
- 4) **场景/域偏移**: 以 file_sourceone-hot 作为域提示很实用, 但若验证集出现**新文件/新域**, 该特征带来的收益会减弱, 且可能产生轻微“按文件记忆”;
- 5) **结构信息未充分利用**: 122 子载波仅做统计切片和 EESM 聚合, 未显式建模**频域相关/选择性衰落形状**; 也未引入时间邻域或 TxBF 信息 (当前仅 TxBF=0)。

8.3 模型的改进与推广

- 1) **更强的 PHY 抽象与标定**: 将 EESM 升级为 **MIESM** 或者分 MCS 的 **EESM β** (按调制/码率或编码块长度分组拟合), 对高阶调制/高码率更准确; 在 3GPP 系统级评估中, MIESM 通常优于 EESM; 把 Isotonic 扩展为分段单调广义回归或“Isotonic+温度/Platt 校准”的级联, 改善尾部外推与平滑性。
- 2) **学习器增强与不确定性**: 采用分位数回归 (HGBR 的 quantile 目标或 LightGBM/XGBoost 的 Quantile) 输出上下分位, 再用**保形预测**构造可解释的置信区间, 便于“保守速率”决策; 尝试 Huber/Quantile 损失以抑制厚尾残差; 保持对 eff_db 的单调约束不变以提升可解释性。
- 3) **结构化表征 122×(Rx×Tx) 的频域形态**: 将 122 维子载波视为**集合输入**, 使用 DeepSets/SetTransformer 这类**置换不变**网络做“频域轮廓”建模, 再与 EESM 特征做浅层融合 (EESM 作为先验/可解释分支)。这样既保留物理先验又增强形状判别力;
- 4) **更严格的泛化与域适配**以 GroupKFold (按 STA/MAC/文件) 作为统一评估协议; 在“新域”上引入

重要性重加权 (IR/KernelMeanMatching) 或 CORAL 做一阶对齐；也可把 file_source 从 one-hot 换为 **域嵌入** 并在训练中使用 **域随机化**；对训练/验证时段相邻样本引入 **时间窗口特征** 与“近邻平滑”决策，降低瞬时噪声对 MCS 选择的抖动。

参考文献

- [1] He X, Niu K, He Z, Lin J. Link Layer Abstraction in MIMO-OFDM System[C]//2007 International Workshop on Cross Layer Design, Jinan, China, 2007.
- [2] Liu H, Cai L, Yang H, Li D. EESM Based Link Error Prediction for Adaptive MIMO-OFDM System[C]//2007 IEEE 65th Vehicular Technology Conference (VTC2007-Spring), Dublin, Ireland, 2007.
- [3] Brueninghaus K, Astely D, Salzer T, Visuri S, Alexiou A, Karger S, Seraji G-A. Link Performance Models for System Level Simulations of Broadband Radio Access Systems[C]//2005 IEEE 16th International Symposium on Personal, Indoor and Mobile Radio Communications (PIMRC 2005), Berlin, Germany, 2005: 2306 – 2311.
- [4] DeepSeek,DeepSeek-R1-0528,深度求索(DeepSeek),2025-09-23
- [5] Zadrozny B , Elkan C . Transforming Classifier Scores into Accurate Multiclass Probability Estimates[C]//Proceedings of the Eighth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD ' 02), Edmonton, Canada, 2002: 694 – 699.
- [6] ChatGPT,GPT-5 Thinking,OpenAI,2025-09-22
- [7] Chepoi V, Cogneau D, Fichet B. Polynomial Algorithms for Isotonic Regression[J]. Lecture Notes – Monograph Series, 1997, 31: 147-160.
- [8] He L, Levine R, Fan J, Beemer J, Stronach J. Random Forest as a Predictive Analytics Alternative to Regression in Institutional Research[J]. Practical Assessment, Research & Evaluation, 2018, 23(1): 1.
- [9] ChatGPT,GPT-5 Thinking,OpenAI,2025-09-23
- [10] Auer E, Ahrens A. Verified Bit and Power Allocation for MIMO Systems: A Comparison of SVD Based Techniques With GMD[J]. Acta Cybern., 2024, 26: 775-798.

附录

附录 1：附件文件及内容

附件	命名
附件 1	完整答案文件夹，均按出题人要求命名
附件 2	各问题源代码

附录 2：问题一关键代码

问题一关键代码	
1	<code># ----- EESM 主体 -----</code>
2	<code>--</code>
3	<code>@dataclass</code>
4	<code>class EESMConfig:</code>
5	<code> """EESM 搜参与回归配置。"""</code>
6	<code> alpha_min: float = 1.0</code>
7	<code> alpha_max: float = 6.0</code>
8	<code> alpha_step: float = 0.2</code>
9	<code> beta_min: float = 0.1</code>
10	<code> beta_max: float = 20.0</code>
11	<code> beta_step: float = 0.5</code>
12	<code> beta_logspace: bool = True</code>
13	<code> n_splits: int = 5</code>
14	<code> clip_to_label: bool = True</code>
15	<code> balance_isotonic: bool = True</code>
16	<code> winsor_eff_db: Optional[Tuple[float, float]] = (-200.0, 200.0)</code>
17	
18	
19	<code>def _stable_log_mean_exp(z: np.ndarray) -> float:</code>
20	<code> """数值稳健的 log(mean(exp(z)))。"""</code>
21	<code> z = np.asarray(z, dtype=float)</code>
22	<code> if z.size == 0:</code>
23	<code> return -np.inf</code>
24	<code> zmax = np.nanmax(z)</code>
25	<code> if not np.isfinite(zmax):</code>
26	<code> zmax = 0.0</code>
27	<code> t = np.exp(np.clip(z - zmax, -745, 709))</code>
28	<code> m = np.nanmean(t)</code>
29	<code> if m <= 0 or not np.isfinite(m):</code>

```

30     m = 1e-300
31     return float(zmax + np.log(m))
32
33
34 def eesm_eff_db_from_seq(sinr_seq: np.ndarray, alpha: float, beta: float
35 ) -> float:
36     """EESM: 线性域 SINR 序列 → 等效 SNR(dB)。
37      $\gamma_i(\text{dB}) = 10 \log_{10}(\text{sinr}_i)$ 
38      $\gamma_{\text{eff}}(\text{dB}) = -\alpha \cdot \log(\text{mean}(\exp(-\gamma_i/\beta)))$ 
39     """
40     gamma_db = 10.0 * np.log10(np.clip(sinr_seq, 1e-300, None))
41     gamma_db = np.clip(gamma_db, -200.0, 200.0)
42     z = -gamma_db / max(beta, 1e-6)
43     lme = _stable_log_mean_exp(z)
44     eff = -float(alpha) * lme
45     if not np.isfinite(eff):
46         eff = float(np.median(gamma_db)) # 兜底: 用中位数
47     return float(np.clip(eff, -300.0, 300.0))
48
49 def _winsorize(x: np.ndarray, lo: Optional[float], hi: Optional[float])
50 -> np.ndarray:
51     if lo is None and hi is None:
52         return x
53     return np.clip(x, lo if lo is not None else -
54 np.inf, hi if hi is not None else np.inf)
55
56
57 def _sample_weights_for_iso(y: np.ndarray) -> np.ndarray:
58     """类别/数值密度倒数加权, 缓解频繁 MCS 的支配效应。"""
59     y_round = np.round(y.astype(float), 3)
60     uniq, inv = np.unique(y_round, return_inverse=True)
61     freq = np.bincount(inv).astype(float)
62     w = 1.0 / (freq[inv] + 1e-12)
63     w *= (len(w) / np.sum(w))
64     return w
65
66
67 def _choose_split(y: np.ndarray, groups: Optional[np.ndarray], n_splits:
68 int):
69     """优先按分组 (如 sta_mac / file_source) 做 GroupKFold; 否则 KFold。"""
70     if groups is not None:
71         ug = np.unique(groups.astype(str))
72         if len(ug) >= 2:

```

```

70         return GroupKFold(n_splits=min(n_splits, len(ug))).split(np.
    zeros_like(y), y, groups)
71     return KFold(n_splits=min(n_splits, max(2, len(y)//10)), shuffle=True
    e, random_state=42).split(np.zeros_like(y), y)
72
73
74 def _cv_score_eesm(X_sinr: np.ndarray, y: np.ndarray, groups: Optional[n
    p.ndarray],
75                     alpha: float, beta: float, cfg: EESMConfig) -> Tuple[
    float, float]:
76     """给定 ( $\alpha, \beta$ ) 的交叉验证分数 (MSE,  $R^2$ )，ISO 作为基线映射。"""
77     fold_mse, fold_r2 = [], []
78     sw_global = _sample_weights_for_iso(y) if cfg.balance_isotonic else
    np.ones_like(y, dtype=float)
79     for tr, va in _choose_split(y, groups, cfg.n_splits):
80         eff_tr = np.array([eesm_eff_db_from_seq(X_sinr[i], alpha, beta)
    for i in tr], dtype=float)
81         eff_va = np.array([eesm_eff_db_from_seq(X_sinr[i], alpha, beta)
    for i in va], dtype=float)
82         if cfg.winsor_eff_db is not None:
83             eff_tr = _winsorize(eff_tr, *cfg.winsor_eff_db)
84             eff_va = _winsorize(eff_va, *cfg.winsor_eff_db)
85             eff_tr = np.nan_to_num(eff_tr, nan=0.0, posinf=300.0, neginf=-
    300.0)
86             eff_va = np.nan_to_num(eff_va, nan=0.0, posinf=300.0, neginf=-
    300.0)
87
88         iso = IsotonicRegression(out_of_bounds="clip")
89         iso.fit(eff_tr, y[tr], sample_weight=sw_global[tr])
90         pred = np.clip(iso.predict(eff_va), np.min(y), np.max(y))
91         fold_mse.append(mean_squared_error(y[va], pred))
92         fold_r2.append(r2_score(y[va], pred))
93     return float(np.mean(fold_mse)), float(np.mean(fold_r2))
94
95
96 def _refine_alpha_beta_random(X_sinr: np.ndarray, y: np.ndarray, groups:
    Optional[np.ndarray],
97                               cfg: EESMConfig, seeds: List[Tuple[float,
    float]],
98                               n_tries: int = 30, a_radius: float = 0.3,
99                               b_scale: Tuple[float, float] = (0.8, 1.25)
    ) -> Dict[str, float]:
100     """以 seeds 为中心做局部随机细化搜索。"""
101     best = {"alpha": None, "beta": None, "mse": np.inf, "r2": -np.inf}

```

```

102     for a0, b0 in seeds:
103         for _ in range(n_tries):
104             a = max(1.0, a0 + np.random.uniform(-a_radius, a_radius))
105             b = max(cfg.beta_min, min(cfg.beta_max, b0 * np.random.unifo
rm(*b_scale)))
106             cv_mse, cv_r2 = _cv_score_eesm(X_sinr, y, groups, a, b, cfg)
107             if (cv_mse + 1e-
12 < best["mse"]) or (abs(cv_mse - best["mse"]) < 1e-
9 and cv_r2 > best["r2"]):
108                 best.update({"alpha": float(a), "beta": float(b), "mse":
float(cv_mse), "r2": float(cv_r2)})
109         return best
110
111
112 def fit_eesm_isotonic(X_sinr: np.ndarray, y: np.ndarray, groups: Optiona
l[np.ndarray], cfg: EESMConfig):
113     """全数据拟合: 粗搜 ( $\alpha, \beta$ )  $\rightarrow$  局部细化  $\rightarrow$  全量 ISO 拟合与训练指标。"""
114     # 网格 (支持线性+对数并集, 提高覆盖度)
115     alphas = np.arange(cfg.alpha_min, cfg.alpha_max + 1e-
9, cfg.alpha_step)
116     if cfg.beta_logspace:
117         betas = np.unique(np.concatenate([
118             np.logspace(np.log10(max(cfg.beta_min, 1e-
2)), np.log10(cfg.beta_max), num=40),
119             np.arange(cfg.beta_min, cfg.beta_max + 1e-9, cfg.beta_step),
120         ]))
121     else:
122         betas = np.arange(cfg.beta_min, cfg.beta_max + 1e-
9, cfg.beta_step)
123
124     best = {"alpha": None, "beta": None, "mse": np.inf, "r2": -np.inf}
125     leaderboard: List[Tuple[float, float, float, float]] = [] # (mse, -
r2, a, b)
126
127     for a in alphas:
128         if a < 1.0:
129             continue
130         for b in betas:
131             cv_mse, cv_r2 = _cv_score_eesm(X_sinr, y, groups, a, b, cfg)
132             if (cv_mse + 1e-
12 < best["mse"]) or (abs(cv_mse - best["mse"]) < 1e-
9 and cv_r2 > best["r2"]):
133                 best.update({"alpha": float(a), "beta": float(b), "mse":
float(cv_mse), "r2": float(cv_r2)})

```

```

134         leaderboard.append((cv_mse, -cv_r2, a, b))
135         if len(leaderboard) > 12:
136             leaderboard = sorted(leaderboard)[:12]
137
138         # 局部随机细化
139         seeds: List[Tuple[float, float]] = [(best["alpha"], best["beta"])]
140         for _, _, a, b in sorted(leaderboard)[:6]:
141             seeds.append((a, b))
142             a0, b0 = best["alpha"], best["beta"]
143             seeds += [(max(1.0, a0 + da), max(cfg.beta_min, min(cfg.beta_max, b0
+ db))) for da in (-0.2, 0.2) for db in (-0.5, 0.5)]
144             best_local = _refine_alpha_beta_random(X_sinr, y, groups, cfg, seeds
)
145             if best_local["mse"] + 1e-12 < best["mse"]:
146                 best = best_local
147
148             alpha, beta = float(best["alpha"]), float(best["beta"])
149
150         # 全量等效 SNR & ISO 拟合
151         eff_full = np.array([eesm_eff_db_from_seq(x, alpha, beta) for x in X
_sinr], dtype=float)
152         if cfg.winsor_eff_db is not None:
153             eff_full = _winsorize(eff_full, *cfg.winsor_eff_db)
154             eff_full = np.nan_to_num(eff_full, nan=0.0, posinf=300.0, neginf=-
300.0)
155
156         iso = IsotonicRegression(out_of_bounds="clip")
157         sw = _sample_weights_for_iso(y) if cfg.balance_isotonic else None
158         iso.fit(eff_full, y, sample_weight=sw)
159
160         y_pred_iso = iso.predict(eff_full)
161         if cfg.clip_to_label:
162             y_pred_iso = np.clip(y_pred_iso, np.min(y), np.max(y))
163
164         metrics = {
165             "alpha": alpha,
166             "beta": beta,
167             "cv_mse": float(best["mse"]),
168             "cv_r2": float(best["r2"]),
169             "train_mse": float(mean_squared_error(y, y_pred_iso)),
170             "train_rmse": float(np.sqrt(mean_squared_error(y, y_pred_iso))),
171             "train_mae": float(mean_absolute_error(y, y_pred_iso)),
172             "train_r2": float(r2_score(y, y_pred_iso)),
173             "n_train": int(len(y)),

```

```

174     }
175     return alpha, beta, iso, eff_full, y_pred_iso, metrics
176
177     # ----- 轻量堆叠 -----
178     -
179
180     def _seq_stats_features(sinr_seq: np.ndarray) -> Dict[str, float]:
181         """从线性域与 dB 域提取少量统计量，兼顾稳健与信息量。"""
182         x = np.asarray(sinr_seq, dtype=float)
183         x = np.clip(x, 1e-12, None)
184         x_db = 10.0 * np.log10(x)
185
186         def pct(v, p):
187             return float(np.percentile(v, p))
188
189         def safe_skew(v):
190             try:
191                 from scipy.stats import skew
192                 return float(skew(v))
193             except Exception:
194                 return 0.0
195
196         def safe_kurt(v):
197             try:
198                 from scipy.stats import kurtosis
199                 return float(kurtosis(v))
200             except Exception:
201                 return 0.0
202
203         feats = {
204             "lin_mean": float(np.mean(x)),
205             "lin_std": float(np.std(x)),
206             "lin_min": float(np.min(x)),
207             "lin_max": float(np.max(x)),
208             "lin_p10": pct(x, 10),
209             "lin_p25": pct(x, 25),
210             "lin_p50": pct(x, 50),
211             "lin_p75": pct(x, 75),
212             "lin_p90": pct(x, 90),
213             "lin_skew": safe_skew(x),
214             "lin_kurt": safe_kurt(x),
215             "db_mean": float(np.mean(x_db)),
216             "db_std": float(np.std(x_db)),
217             "db_min": float(np.min(x_db)),

```

```

217         "db_max": float(np.max(x_db)),
218         "db_p10": pct(x_db, 10),
219         "db_p25": pct(x_db, 25),
220         "db_p50": pct(x_db, 50),
221         "db_p75": pct(x_db, 75),
222         "db_p90": pct(x_db, 90),
223         "share_above_1": float(np.mean(x > 1.0)),
224         "share_above_3": float(np.mean(x > 3.0)),
225         "share_above_10": float(np.mean(x > 10.0)),
226     }
227     return feats
228
229
230 def _eesm_on_slice(x: np.ndarray, a: float, b: float) -> float:
231     """在一个子区间上计算等效 SNR(dB)（用于分段统计）。"""
232     gamma_db = 10.0 * np.log10(np.clip(x, 1e-300, None))
233     gamma_db = np.clip(gamma_db, -200.0, 200.0)
234     z = -gamma_db / max(b, 1e-6)
235     z_max = np.max(z)
236     m = np.mean(np.exp(np.clip(z - z_max, -745, 709)))
237     lme = z_max + np.log(m + 1e-300)
238     return float(np.clip(-a * lme, -300.0, 300.0))
239
240
241 def build_meta_features(X_sinr: np.ndarray, eff_db: np.ndarray, noise_db
242 m: np.ndarray,
243                        alpha: float, beta: float, files: np.ndarray) ->
244 pd.DataFrame:
245     """构造二级学习器所需的元特征（少量且可解释）。"""
246     rows: List[Dict[str, float]] = []
247     K_ = X_sinr.shape[1]
248     cuts = [0, K_//3, 2*K_//3, K_]
249     uniq_files = np.unique(files)
250     for i in range(len(X_sinr)):
251         seq = X_sinr[i]
252         f = _seq_stats_features(seq)
253         e1 = _eesm_on_slice(seq[cuts[0]:cuts[1]], alpha, beta)
254         e2 = _eesm_on_slice(seq[cuts[1]:cuts[2]], alpha, beta)
255         e3 = _eesm_on_slice(seq[cuts[2]:cuts[3]], alpha, beta)
256         f.update({
257             "eff_db": float(eff_db[i]),
258             "phi_n1": float(np.log1p(np.power(10.0, eff_db[i]/10.0))), #
259             "eff_seg1": e1,

```

```

258         "eff_seg2": e2,
259         "eff_seg3": e3,
260         "eff_seg_range": float(max(e1, e2, e3) - min(e1, e2, e3)),
261         "eff_seg_var": float(np.var([e1, e2, e3])),
262         "noise_dbm": float(noise_dbm[i]),
263     })
264     # one-hot 文件来源, 避免不同采集批次的偏移
265     for fname in uniq_files:
266         f[f"file_{fname}"] = 1.0 if files[i] == fname else 0.0
267     rows.append(f)
268     df = pd.DataFrame(rows).replace([np.inf, -
np.inf], np.nan).fillna(0.0)
269     cols = list(df.columns)
270     cols = ["eff_db"] + [c for c in cols if c != "eff_db"] # 将 eff_db
放在首列 (后续可单调约束)
271     return df[cols]
272
273
274 def _oof_iso_predict(X_sinr: np.ndarray, y: np.ndarray, groups: Optional
[np.ndarray],
275                     alpha: float, beta: float, cfg: EESMConfig) -> np.n
darray:
276     """生成 ISO 的 OOF 预测 (避免信息泄露的融合权重估计)。"""
277     oof = np.zeros_like(y, dtype=float)
278     for tr, va in _choose_split(y, groups, cfg.n_splits):
279         eff_tr = np.array([eesm_eff_db_from_seq(X_sinr[i], alpha, beta)
for i in tr], dtype=float)
280         eff_va = np.array([eesm_eff_db_from_seq(X_sinr[i], alpha, beta)
for i in va], dtype=float)
281         if cfg.winsor_eff_db is not None:
282             eff_tr = _winsorize(eff_tr, *cfg.winsor_eff_db)
283             eff_va = _winsorize(eff_va, *cfg.winsor_eff_db)
284         iso = IsotonicRegression(out_of_bounds="clip")
285         w = _sample_weights_for_iso(y[tr]) if cfg.balance_isotonic else
None
286         iso.fit(eff_tr, y[tr], sample_weight=w)
287         oof[va] = np.clip(iso.predict(eff_va), np.min(y), np.max(y))
288     return oof
289
290
291 def _fit_hgbr_stack(X_feat: pd.DataFrame, y: np.ndarray, monotone_for_ef
f: bool = True):
292     """小网格搜索 HGBR; 对 eff_db 施加单调递增约束以增强可解释性。"""
293     mono = None

```

```

294     if monotone_for_eff:
295         try:
296             mono = [1] + [0] * (X_feat.shape[1] - 1) # 第一列 eff_db 单调
                递增
297         except Exception:
298             mono = None
299
300     space = [
301         {"max_leaf_nodes": 31, "learning_rate": 0.08, "max_iter": 300, "
l2_regularization": 0.01},
302         {"max_leaf_nodes": 63, "learning_rate": 0.06, "max_iter": 400, "
l2_regularization": 0.02},
303         {"max_leaf_nodes": 31, "learning_rate": 0.10, "max_iter": 250, "
l2_regularization": 0.00},
304     ]
305     best_model, best_cfg, best_r2 = None, None, -np.inf
306     for cfg in space:
307         kw = dict(loss="squared_error", max_leaf_nodes=cfg["max_leaf_nod
es"],
308                 learning_rate=cfg["learning_rate"], max_iter=cfg["max_
iter"],
309                 l2_regularization=cfg["l2_regularization"], early_stop
ping=True,
310                 validation_fraction=0.1)
311         if mono is not None:
312             kw["monotonic_cst"] = mono
313         model = HistGradientBoostingRegressor(**kw)
314         model.fit(X_feat, y)
315         r2 = r2_score(y, model.predict(X_feat))
316         if r2 > best_r2:
317             best_model, best_cfg, best_r2 = model, cfg, r2
318     return best_model, best_cfg
319
320
321 def _best_blend_weight(y: np.ndarray, iso_pred: np.ndarray, gbdt_pred: n
p.ndarray) -> float:
322     """按最小化融合后误差方差的闭式权重估计（投影解）。"""
323     d = gbdt_pred - iso_pred
324     v = np.var(d)
325     if v < 1e-12:
326         return 0.5
327     w = np.mean((y - iso_pred) * d) / v
328     return float(np.clip(w, 0.0, 1.0))

```

附录 3：问题二关键代码

问题二关键代码

```

1      # ===== 特征构建 =====
2
3      def _safe_skew(x: np.ndarray) -> float:
4          x = np.asarray(x, dtype=float)
5          if x.size == 0: return 0.0
6          mu, sd = float(np.mean(x)), float(np.std(x) + 1e-12)
7          return float(np.mean(((x - mu) / sd) ** 3))
8
9      def _lin_trend(y: np.ndarray) -> float:
10         if y.size < 2: return 0.0
11         idx = np.arange(y.size, dtype=float)
12         try: return float(np.polyfit(idx, y, 1)[0])
13         except Exception: return 0.0
14
15     def _augment_features_from_seq(seq: np.ndarray) -> np.ndarray:
16         """少量、可解释的统计增强（线性域 + dB 域）。"""
17         s = np.asarray(seq, dtype=float)
18         sdb = 10.0 * np.log10(np.clip(s, 1e-12, None))
19         lin = [np.mean(s), np.std(s), np.percentile(s, 10), np.percentile(s,
20             50), np.percentile(s, 90), _safe_skew(s)]
21         db = [np.mean(sdb), np.std(sdb), np.percentile(sdb, 10), np.percentile(sdb, 50), np.percentile(sdb, 90),
22             np.percentile(sdb, 75) - np.percentile(sdb, 25), _safe_skew(sdb), _lin_trend(sdb),
23             (np.std(np.diff(sdb)) if sdb.size > 1 else 0.0)]
24         return np.array(lin + db, dtype=float)
25
26     AUG_FEATURE_NAMES = [
27         # 线性域统计 (6)
28         "lin_mean", "lin_std", "lin_p10", "lin_p50", "lin_p90", "lin_skew",
29         # dB 域统计 (9)
30         "db_mean", "db_std", "db_p10", "db_p50", "db_p90", "db_iqr", "db_skew", "db_trend", "db_diff_std"
31     ]
32
33     def get_feature_names() -> List[str]:
34         base_lin = [f"SINR_lin[k{ki}]" for ki in range(K)]
35         if USE_FEATURE_AUG:
36             base_db = [f"SINR_db[k{ki}]" for ki in range(K)]
37         return base_lin + base_db + AUG_FEATURE_NAMES

```

```

38     return base_lin
39
40 def build_train_matrix(df_train: pd.DataFrame):
41     """从训练表构建模型矩阵: X、y、分组、文件名等。仅保留 TxBF=0。"""
42     sub = df_train[df_train["beamforming_en"] == 0].copy()
43     if sub.empty:
44         raise RuntimeError("训练集中 beamforming_en==0 的样本为 0")
45
46     X, y, groups, files = [], [], [], []
47     for _, row in sub.iterrows():
48         seq = sinr_sequence_for_row(row)
49         if USE_FEATURE_AUG:
50             feat = np.concatenate([seq, 10*np.log10(np.clip(seq, 1e-
12, None)), _augment_features_from_seq(seq)])
51         else:
52             feat = seq
53         X.append(feat)
54         y.append(float(row["mcs"]))
55         groups.append(row.get("sta_mac", row.get("file_source", "NA")))
56         files.append(row.get("file_source", "NA"))
57     return np.vstack(X), np.array(y), np.array(groups, dtype=str), np.ar
ray(files, dtype=str)
58
59 def build_valid_matrix(df_valid: pd.DataFrame):
60     """构建验证集特征矩阵与辅助信息（保持与训练期特征一致）。"""
61     sub = df_valid[df_valid["beamforming_en"] == 0].copy()
62     Xv, idx, files = [], [], []
63     for i, row in sub.iterrows():
64         seq = sinr_sequence_for_row(row)
65         if USE_FEATURE_AUG:
66             feat = np.concatenate([seq, 10*np.log10(np.clip(seq, 1e-
12, None)), _augment_features_from_seq(seq)])
67         else:
68             feat = seq
69         Xv.append(feat)
70         idx.append(i)
71         files.append(row.get("file_source", "NA"))
72     if len(Xv) == 0:
73         return np.empty((0, len(get_feature_names()))), np.array([], dtype=int), np.array([], dtype=str)
74     return np.vstack(Xv), np.array(idx, dtype=int), np.array(files, dtype=str)
75     # ===== 交叉验证 & 调参 =====
76

```

```

77 def make_splitter(y: np.ndarray, groups: Optional[np.ndarray], n_splits:
    int = 5):
78     if groups is not None and len(np.unique(groups)) >= 2:
79         return GroupKFold(n_splits=min(n_splits, len(np.unique(groups)))
    ).split(np.zeros_like(y), y, groups)
80     return KFold(n_splits=min(n_splits, max(2, len(y)//10)), shuffle=True,
    random_state=42).split(np.zeros_like(y), y)
81
82 def rf_default_params() -> Dict:
83     return dict(
84         n_estimators=500,
85         max_depth=None,
86         random_state=1,
87         n_jobs=N_JOBS_FINAL, # 评估与最终训练皆单线程
88         max_features="sqrt",
89         min_samples_leaf=2,
90         bootstrap=True,
91         oob_score=False,
92     )
93
94 def cv_evaluate_rf(X: np.ndarray, y: np.ndarray, groups: Optional[np.nda
    rray], n_splits: int,
95         label_set_for_mapping: Optional[np.ndarray], rf_para
    ms: Optional[Dict] = None,
96         use_isotonic_in_fold: bool = True, use_inverse_freq_
    weight: bool = True) -> Dict:
97     """交叉验证：同时给出连续(raw)与贴标签(mapped)两套指标，口径与提交一致。
    """
98     base = rf_default_params()
99     if rf_params:
100         base.update(rf_params)
101     # 仅传入 RandomForestRegressor 支持的参数
102     rf_proto = RandomForestRegressor()
103     clean_params = {k: v for k, v in base.items() if k in rf_proto.get_p
    arams()}
104
105     raw_mse, raw_mae, raw_r2 = [], [], []
106     map_mse, map_mae, map_r2 = [], [], []
107
108     for tr, va in make_splitter(y, groups, n_splits):
109         model = RandomForestRegressor(**clean_params)
110         if use_inverse_freq_weight:
111             counts = pd.Series(y[tr]).value_counts()

```

```

112         w_tr = np.array([1.0 / counts.get(v, 1.0) for v in y[tr]], d
type=float)
113         w_tr *= (len(w_tr) / w_tr.sum())
114         model.fit(X[tr], y[tr], sample_weight=w_tr)
115     else:
116         model.fit(X[tr], y[tr])
117
118     p_va = model.predict(X[va])
119     if use_isotonic_in_fold:
120         ir = IsotonicRegression(out_of_bounds="clip")
121         p_tr = model.predict(X[tr])
122         ir.fit(p_tr, y[tr])
123         p_va = ir.predict(p_va)
124
125     raw_mse.append(mean_squared_error(y[va], p_va))
126     raw_mae.append(mean_absolute_error(y[va], p_va))
127     raw_r2.append(r2_score(y[va], p_va))
128
129     if label_set_for_mapping is not None and len(label_set_for_mappi
ng) > 0:
130         p_map = map_to_nearest_label(p_va, np.asarray(label_set_for_
mapping, dtype=float))
131         map_mse.append(mean_squared_error(y[va], p_map))
132         map_mae.append(mean_absolute_error(y[va], p_map))
133         map_r2.append(r2_score(y[va], p_map))
134
135     out = {
136         "cv_mse_raw": float(np.mean(raw_mse)),
137         "cv_rmse_raw": float(np.sqrt(np.mean(raw_mse))),
138         "cv_mae_raw": float(np.mean(raw_mae)),
139         "cv_r2_raw": float(np.mean(raw_r2)),
140         "folds": int(n_splits),
141         "rf_params_used": clean_params,
142     }
143     if map_mse:
144         out.update({
145             "cv_mse_mapped": float(np.mean(map_mse)),
146             "cv_rmse_mapped": float(np.sqrt(np.mean(map_mse))),
147             "cv_mae_mapped": float(np.mean(map_mae)),
148             "cv_r2_mapped": float(np.mean(map_r2)),
149         })
150     return out
151

```

```

152 def tune_rf(X: np.ndarray, y: np.ndarray, groups: Optional[np.ndarray],
153            n_iter: int = TUNE_NITER) -> Tuple[Dict, float]:
154     """轻量随机搜索（可选），评分用  $R^2$ ，返回最优参数及其分数。"""
155     base = RandomForestRegressor(**rf_default_params())
156     param_dist = {
157         "n_estimators": [400, 500, 600, 800],
158         "max_depth": [None, 12, 16, 20],
159         "min_samples_split": [2, 5, 10],
160         "min_samples_leaf": [2, 4, 8, 12, 16],
161         "max_features": ["sqrt", 0.3, 0.5, 0.7],
162         "bootstrap": [True],
163     }
164     # 组折优先
165     if groups is not None and len(np.unique(groups)) >= 2:
166         cv = GroupKFold(n_splits=min(5, len(np.unique(groups))))
167         cv_split = cv.split(X, y, groups)
168     else:
169         cv = KFold(n_splits=min(5, max(2, len(y)//10)), shuffle=True, random_state=42)
170         cv_split = cv.split(X, y)
171     search = RandomizedSearchCV(
172         estimator=base,
173         param_distributions=param_dist,
174         n_iter=n_iter,
175         scoring="r2",
176         cv=list(cv_split),
177         random_state=42,
178         n_jobs=-1,          # 调参阶段可用多核
179         verbose=1,
180         refit=True,
181     )
182     search.fit(X, y)
183     return search.best_params_, float(search.best_score_)
184
185     # ===== 训练、保存、可视化 =====
186
187 def map_to_nearest_label(pred_cont: np.ndarray, label_set: np.ndarray) -
188     > np.ndarray:
189     """将连续预测贴近到训练标签集合的最近邻（浮点标签）。"""
190     label_arr = np.asarray(label_set, dtype=float).reshape(1, -
191     1) # (1, L)
192     pred_arr = np.asarray(pred_cont, dtype=float).reshape(-
193     1, 1) # (N, 1)

```

```

191     d = np.abs(pred_arr - label_arr) # (N, L)
192     idx = np.argmin(d, axis=1)
193     return np.asarray(label_set, dtype=float)[idx]
194
195 def fit_rf_full(X: np.ndarray, y: np.ndarray, train_label_set: np.ndarray,
196               rf_params: Optional[Dict] = None):
197     """全量训练: RF + Isotonic 校准; 保存模型包与训练指标。"""
198     params = rf_default_params()
199     if rf_params: params.update(rf_params)
200     # 仅保留有效参数键
201     rf_proto = RandomForestRegressor()
202     clean = {k: v for k, v in params.items() if k in rf_proto.get_params(
203         ())}
204
205     rf = RandomForestRegressor(**clean)
206     # 逆频样本权重: 缓解常见标签的支配效应
207     counts = pd.Series(y).value_counts()
208     w = np.array([1.0 / counts.get(v, 1.0) for v in y], dtype=float)
209     w *= (len(w) / w.sum())
210
211     rf.fit(X, y, sample_weight=w)
212     yhat = rf.predict(X)
213
214     iso = IsotonicRegression(out_of_bounds="clip")
215     iso.fit(yhat, y)
216     yhat_cal = iso.predict(yhat)
217
218     train_info = {
219         "train_mse": float(mean_squared_error(y, yhat_cal)),
220         "train_rmse": float(np.sqrt(mean_squared_error(y, yhat_cal))),
221         "train_mae": float(mean_absolute_error(y, yhat_cal)),
222         "train_r2": float(r2_score(y, yhat_cal)),
223         "rf_params": clean,
224     }
225
226     model_pack = {
227         "model": rf,
228         "rf_iso": iso, # RF 输出的单调校准器
229         "label_set": np.asarray(sorted(train_label_set), dtype=float),
230         "mcs_min": float(np.min(y)),
231         "mcs_max": float(np.max(y)),
232     }
233
234     with open(OUT/"models"/"rf_q2_model.pkl", "wb") as f:

```

```

233         pickle.dump(model_pack, f)
234         with open(OUT/"tables"/"rf_train_metrics.json", "w", encoding="utf-
            8") as f:
235             json.dump(train_info, f, ensure_ascii=False, indent=2)
236
237         return model_pack, yhat_cal, train_info

```

附录 4：问题三关键代码

问题三关键代码

```

1  # ===== EESM( $\beta$ ) + Isotonic =====
   =====
2  def eesm_eff_db_from_seq(sinr_seq: np.ndarray, beta_db: float) -> float:
3      """EESM:  $\gamma_{\text{eff}}(\text{dB}) = -\beta \cdot \log(\text{mean}(\exp(-\gamma_k/\beta)))$ , 其
        中  $\gamma_k = 10 \log_{10}(\text{SINR}_k)$ 。"""
4      gamma_db = 10.0 * np.log10(np.clip(sinr_seq, 1e-12, None))
5      tmp = np.exp(-gamma_db / max(beta_db, 1e-6))
6      return float(-beta_db * np.log(np.mean(tmp)))
7
8  @dataclass
9  class EESMConfig:
10     beta_min: float = 1.0
11     beta_max: float = 20.0
12     beta_step: float = 0.5
13     refine_low: Optional[float] = 16.0
14     refine_high: Optional[float] = 19.0
15     refine_step: float = 0.1
16     n_splits: int = 5
17     clip_to_label: bool = True
18
19     def _get_split(y: np.ndarray, groups: Optional[np.ndarray], n_splits: in
        t):
20         if groups is not None and len(np.unique(groups)) >= 2:
21             return GroupKFold(n_splits=min(n_splits, len(np.unique(groups)))
                ).split(np.zeros_like(y), y, groups)
22         return KFold(n_splits=min(n_splits, max(2, len(y)//10)), shuffle=True,
                random_state=42).split(np.zeros_like(y), y)
23
24     def fit_eesm_isotonic_txbf(X_sinr: np.ndarray, y: np.ndarray, groups: Op
        tional[np.ndarray], cfg: EESMConfig):
25         """扫描  $\beta$  (粗/细), 用 Isotonic 学习单调映射, 返回最佳  $\beta$  与训练指标。"""
26         gamma_db_all = 10.0 * np.log10(np.clip(X_sinr, 1e-12, None)) # (N, K)

```

```

27     splits = list(_get_split(y, groups, cfg.n_splits))
28
29     def eff_from_beta(beta: float) -> np.ndarray:
30         tmp = np.exp(-gamma_db_all / max(beta, 1e-6)) # (N,K)
31         return -beta * np.log(np.mean(tmp, axis=1)) # (N,)
32
33     best_beta, best_mse, beta_curve = None, np.inf, []
34
35     for beta in np.arange(cfg.beta_min, cfg.beta_max + 1e-
36 9, cfg.beta_step):
37         eff = eff_from_beta(float(beta))
38         fold_mse = []
39         for tr, va in splits:
40             iso = IsotonicRegression(out_of_bounds="clip").fit(eff[tr],
41 y[tr])
42             fold_mse.append(mean_squared_error(y[va], iso.predict(eff[va
43 ])))
44         mse_beta = float(np.mean(fold_mse)); beta_curve.append((float(be
45 ta), mse_beta))
46         if mse_beta < best_mse: best_mse, best_beta = mse_beta, float(be
47 ta)
48
49     if (cfg.refine_low is not None) and (cfg.refine_high is not None):
50         for beta in np.arange(cfg.refine_low, cfg.refine_high + 1e-
51 9, cfg.refine_step):
52             eff = eff_from_beta(float(beta))
53             fold_mse = []
54             for tr, va in splits:
55                 iso = IsotonicRegression(out_of_bounds="clip").fit(eff[t
56 r], y[tr])
57                 fold_mse.append(mean_squared_error(y[va], iso.predict(ef
58 f[va])))
59             mse_beta = float(np.mean(fold_mse)); beta_curve.append((floa
60 t(beta), mse_beta))
61             if mse_beta < best_mse: best_mse, best_beta = mse_beta, floa
62 t(beta)
63
64     eff_full = eff_from_beta(best_beta)
65     iso_full = IsotonicRegression(out_of_bounds="clip").fit(eff_full, y)
66     y_pred = iso_full.predict(eff_full)
67     if cfg.clip_to_label:
68         y_pred = np.clip(y_pred, np.min(y), np.max(y))
69
70     metrics = {

```

```

61         "beta_db": best_beta,
62         "train_mse": float(mean_squared_error(y, y_pred)),
63         "train_rmse": float(np.sqrt(mean_squared_error(y, y_pred))),
64         "train_mae": float(mean_absolute_error(y, y_pred)),
65         "train_r2": float(r2_score(y, y_pred)),
66     }
67     return best_beta, iso_full, eff_full, y_pred, metrics, beta_curve
68
69     # ===== 特征工程
70     (TxBF=1) =====
71     def _safe_skew(x: np.ndarray) -> float:
72         x = np.asarray(x, dtype=float)
73         if x.size == 0: return 0.0
74         mu, sd = float(np.mean(x)), float(np.std(x) + 1e-12)
75         return float(np.mean((x - mu) / sd ** 3))
76
77     def _lin_slope(y: np.ndarray) -> float:
78         if y.size < 2: return 0.0
79         idx = np.arange(y.size, dtype=float)
80         try: return float(np.polyfit(idx, y, 1)[0])
81         except Exception: return 0.0
82
83     def _eesm_multi_db(snr_seq: np.ndarray, beta_grid: np.ndarray) -> List[
84         float]:
85         return [eesm_eff_db_from_seq(snr_seq, float(b)) for b in beta_grid]
86
87     def _svd_stats_from_row(row: pd.Series) -> Tuple[float, float, float, fl
88         oat]:
89         """以闭式向量化得到 SVD 结构统计: max/2nd 奇异值均值、条件数均值、波束赋形
90         增益均值。"""
91         arrays = [parse_csi_cell(row[f"csi_matrix_r{r}_c{c}"]) for r in rang
92             e(NRX) for c in range(NTX)]
93         A = np.stack(arrays, axis=0)
94         H1, H2 = A[0:4, :], A[4:8, :]
95         a = np.sum(np.abs(H1)**2, axis=0)
96         d = np.sum(np.abs(H2)**2, axis=0)
97         b = np.sum(H1 * np.conj(H2), axis=0)
98         rad = np.sqrt((a - d)**2 + 4.0 * (np.abs(b)**2))
99         lam_max = 0.5 * (a + d + rad)
100        lam_min = 0.5 * (a + d - rad)
101        s1 = np.sqrt(np.clip(lam_max, 0, None))
102        s2 = np.sqrt(np.clip(lam_min, 0, None))
103        cond = s1 / (np.maximum(s2, 1e-12))
104        gain = lam_max

```

```

100     return float(np.mean(s1)), float(np.mean(s2)), float(np.mean(cond)),
        float(np.mean(gain))
101
102 def _row_features_txbf(row: pd.Series, beta_grid: np.ndarray) -> Tuple[n
    p.ndarray, float, np.ndarray]:
103     """构造 ML 使用的特征: 线性/ dB 统计 + SVD 统计 + 多 $\beta$  EESM; 返回 (特征向
        量,  $\gamma$  均值(dB), SINR 序列)"""
104     sinr = sinr_sequence_for_row_txbf(row)
105     gdb = 10.0 * np.log10(np.clip(sinr, 1e-12, None))
106
107     lin = [np.mean(sinr), np.std(sinr), np.percentile(sinr, 10), np.perc
        entile(sinr, 50), np.percentile(sinr, 90), _safe_skew(sinr)]
108     db = [np.mean(gdb), np.std(gdb), np.percentile(gdb, 10), np.perce
        ntile(gdb, 50), np.percentile(gdb, 90),
109         (np.percentile(gdb, 75) - np.percentile(gdb, 25)), _safe_skew
        (gdb), _lin_slope(gdb), (np.std(np.diff(gdb)) if gdb.size>1 else 0.0)]
110
111     s1m, s2m, condm, gainm = _svd_stats_from_row(row)
112     eesm_feats = _eesm_multi_db(sinr, beta_grid)
113
114     noise_dbm = float(row["noise_floor"]) # 有时对数据批次存在轻微偏置
115     feats = lin + db + [s1m, s2m, condm, gainm, noise_dbm] + eesm_feats
116     return np.array(feats, dtype=float), float(np.mean(gdb)), sinr
117
118 def build_sinr_matrix_txbf(df_sub: pd.DataFrame) -> Tuple[np.ndarray, np
    .ndarray, np.ndarray, np.ndarray]:
119     X, y, groups, ge_db, files = [], [], [], [], []
120     for _, row in df_sub.iterrows():
121         seq = sinr_sequence_for_row_txbf(row)
122         X.append(seq)
123         y.append(float(row["mcs"]))
124         groups.append(row.get("sta_mac", row.get("file_source", "NA")))
125         ge_db.append(10.0 * np.log10(np.mean(seq)))
126         files.append(row.get("file_source", "NA"))
127     return np.vstack(X), np.array(y), np.array(groups, dtype=str), np.ar
        ray(ge_db), np.array(files, dtype=str)
128
129 def build_ml_feature_matrix_txbf(df_sub: pd.DataFrame, beta_grid: np.nda
    rray, best_beta: Optional[float]=None):
130     Xf, y, groups, files, ge_db, eff_best = [], [], [], [], [], []
131     for _, row in df_sub.iterrows():
132         feats, ge, sinr = _row_features_txbf(row, beta_grid)
133         if best_beta is not None:
134             effb = eesm_eff_db_from_seq(sinr, float(best_beta))

```

```
135         feats = np.append(feats, effb); eff_best.append(effb)
136     else:
137         eff_best.append(np.nan)
138     Xf.append(feats)
139     y.append(float(row["mcs"]))
140     groups.append(row.get("sta_mac", row.get("file_source", "NA")))
141     files.append(row.get("file_source", "NA"))
142     ge_db.append(ge)
143     return (
144         np.vstack(Xf), np.array(y), np.array(groups, dtype=str), np.array(
145             files, dtype=str), np.array(ge_db), np.array(eff_best)
```
